



UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO

DEPARTMENT OF COMPUTER SCIENCE

MASTER'S DEGREE IN COMPUTER SCIENCE

MASTER'S THESIS
IN
SOFTWARE ENGINEERING

An Agentic AI Framework for Regulatory Documentation of AI-Based Medical Software

SUPERVISORS:

Prof. Giulio Mallardi
Prof. Filippo Lanubile

CANDIDATE:

Fabrizio Rosmarino

ACADEMIC YEAR 2025 - 2026

*A chi ha imparato a cavarsela da solo e a chi
sta ancora cercando il proprio modo di farlo.*

*A chi si è sentito smarrito, ha avuto paura, ha
dubitato, ma ha scelto comunque di continuare.*

*A chi custodisce una parte fragile come il
cristallo e una forza capace di resistere come
cemento armato.*

*A chi non ha sempre conosciuto la direzione,
ma ha continuato a cercarla.*

*Perché forse non serve conoscere la strada fin
dall'inizio. Serve soltanto il coraggio di
percorrerla abbastanza a lungo da farla
diventare propria.*

F.R.

*Alla mia famiglia, per avermi dato la forza di
andare avanti e il coraggio di spingermi oltre i
miei limiti.*

*A Martina, per essermi stata accanto con una
presenza costante, paziente e sincera.*

*Ai miei amici di sempre e a quelli incontrati
lungo il cammino, per aver dato un volto ai
ricordi più belli di questi anni.*

*A chi da lassù continua a camminare al mio
fianco.*

*A me stesso, per aver continuato a credere nei
miei obiettivi con costanza e determinazione.*

Disclaimer

All trademarks, trade names, products, and logos mentioned in this thesis are the property of their respective owners. The author does not claim any ownership rights over these trademarks or trade names. They are used solely for informational and descriptive purposes and without any intention of infringement.



Questa tesi è stata realizzata nell'ambito delle attività di ricerca del

Collab – Collaborative Development Group
<https://collab.di.uniba.it/>

Dipartimento di Informatica, Università degli Studi di Bari Aldo Moro

Contents

Introduction	1
1 Medical Software and the European Regulatory Framework	4
1.1 Software as a Medical Device	4
1.1.1 The Regulatory Definition of a Medical Device	4
1.1.2 Qualification of Software: MDSW, SiMD and SaMD	5
1.1.3 AI-Based SaMD	5
1.2 The Regulatory Framework for Medical Devices in the European Union	7
1.2.1 Risk Classification under the MDR	7
1.2.2 The Relevant Technical Standards	10
1.3 The European Regulation on Artificial Intelligence	11
1.3.1 AI-Based Medical Software as a High-Risk System	11
1.3.2 Documentary Obligations	12
1.3.3 Data Quality and Governance	12
1.4 Technical Documentation as a Software Artefact	13
2 Language Models, Retrieval, and Agents	15
2.1 Large Language Models	15
2.2 Retrieval-Augmented Generation	16
2.3 Corrective RAG	17
2.4 LLM-Based Agentic Systems	18
2.4.1 Reasoning and Acting	18
2.4.2 State-Graph Orchestration	19
3 Existing Approaches	21
3.1 Life-Cycle Automation: from DevOps to MLOps	21
3.2 RegOps and Compliance as Code	22
3.3 Technical Documentation for the AI Act	23
3.4 LLM Agents and Behavioural Control	24
3.5 RAG in Regulated Domains	25
3.6 The Starting Point	25

4	Data Documentation: Fields and Strategies	27
4.1	From the Template to the Field Model	27
4.2	Where the Information Resides	28
4.3	The Three Strategies	29
4.4	Application to the Data Documentation	30
4.5	The Admissible Outcomes	33
5	An Agentic Framework for Data Documentation	35
5.1	The Architecture	35
5.2	The Field Specification	37
5.3	Initialization and Shared State	38
5.4	Evidence Gathering	39
5.4.1	Observation Tools	40
5.4.2	The Retrieval Tool	40
5.5	Evidence Grading and Abstention	41
5.6	Controlled Generation and Faithfulness Verification	42
5.7	Compilation, Provenance and Audit Trail	44
5.8	Implementation	45
5.9	Case Study	46
5.10	Research Questions	46
5.11	Ground Truth	47
5.12	Experimental Settings	48
5.13	Evaluation metrics	49
5.14	Final Results	52
5.15	Comparison Across Model Families	56
	Conclusions and Future Work	61
	Bibliography	65
	Acknowledgements	71

Introduction

Artificial intelligence is increasingly used in medical software for tasks such as diagnosis, prediction, monitoring and decision support. In the European Union, these systems are subject to a regulatory framework in which conformity is demonstrated not only through the technical quality of the software, but also through the documentation that describes its design, development, validation, risks and data.

These requirements become particularly demanding for AI-based medical software. This type of software must remain traceable, controlled and up to date throughout its life cycle. Both the Medical Device Regulation and the AI Act require technical documentation to be maintained over time. Part of the information required by this documentation is already available in software repositories, configuration files, version-control history, data-versioning artefacts and technical reports. Other information may exist only in discursive form, while some fields depend on human judgement and cannot be derived directly from project artefacts.

A system that automatically produces regulatory documentation must determine, at the same time, how each piece of information can be obtained: whether it can be extracted directly, recovered from textual evidence, or should not be generated automatically. This distinction is especially important when language models are involved. Although they can produce coherent technical text, they may also generate plausible statements that are not supported by the available evidence. In a regulatory context, this is particularly problematic because unsupported content may appear valid during external review, whereas an unresolved field makes the absence of information visible.

Existing work on continuous compliance, including RegOps and automated documentation, has shown that regulatory artefacts can be integrated into software development workflows. At the same time, retrieval-augmented generation and

Corrective RAG provide mechanisms for grounding generated text in retrieved evidence and for assessing whether that evidence is sufficient. Agentic approaches add a further layer by allowing the system to choose among different actions during execution.

This thesis addresses this problem through an agentic framework for the generation of Data Documentation for AI-based medical software. The framework associates each document field with one of three strategies. Observable facts are handled by deterministic extraction tools; discursive information is retrieved from a closed corpus built from the repository and evaluated before generation; information that requires judgement or organisational knowledge is left to human input. For retrieval-based fields, generation is allowed only when the available evidence is considered sufficient, and the resulting text is subsequently checked for faithfulness. The framework is designed so that provenance is preserved throughout the process. Deterministic values are associated with the artefacts from which they were extracted, retrieved fragments retain their source location, and the final document is accompanied by an audit trail describing how each field was produced. The objective is therefore not only to automate document production, but to make the resulting documentation inspectable and verifiable.

The thesis is organised as follows:

- **Chapter 1** introduces Software as a Medical Device, the relevant European regulatory framework and the documentary requirements that motivate the work.
- **Chapter 2** presents the technical strategy based on language models, retrieval-augmented generation, Corrective-RAG and agentic systems.
- **Chapter 3** discusses related works about MLOps, RegOps, regulatory documentation and controlled LLM-based systems.
- **Chapter 4** defines the field model and the strategies associated with the Data Documentation template.
- **Chapter 5** presents the proposed architecture (structure, components, behaviour and implementation) and evaluates the proposed framework on a real case through an ablation study and a comparison across different model families.
- **The final Chapter** discusses the main results, limitations and possible future work.

Chapter 1

Medical Software and the European Regulatory Framework

1.1 Software as a Medical Device

1.1.1 The Regulatory Definition of a Medical Device

Regulation (EU) 2017/745 on medical devices, hereafter the MDR, defines a medical device as any instrument, apparatus, appliance, *software*, implant, reagent, material or other article that the manufacturer intends to be used on human beings, alone or in combination, for one or more specific medical purposes, among which the diagnosis, prevention, monitoring, prediction, prognosis, treatment or alleviation of disease [1].

The presence of software in the definition of a medical device establishes that an artefact without a physical component can produce direct clinical effects, and it establishes that software is subject to the same obligations of safety, control and traceability laid down for material devices. Those obligations translate, to a large extent, into the production and maintenance of the regulatory documentation used for final certification.

A second element of the definition deserves to be isolated: the role assigned to the *intended purpose*, that is, the use for which the device is intended according to the information supplied by the manufacturer. The qualification of an artefact as a medical device does not follow from its technical characteristics, but from the

purpose the manufacturer assigns to it: two functionally identical pieces of software may fall under different regulatory regimes depending on what the manufacturer declares.

1.1.2 Qualification of Software: MDSW, SiMD and SaMD

Guidance document MDCG 2019-11 defines **Medical Device Software** (MDSW) as software intended to be used, alone or in combination, for one of the purposes set out in the definition of a medical device in the MDR or the IVDR, irrespective of whether it operates independently or drives or influences the use of a device [2]. This covers both standalone software and software embedded in the hardware of a device.

On this basis it has become customary to distinguish two subsets according to the relationship with the hosting hardware. *Embedded* software that meets the definition of a medical device is termed **Software in a Medical Device** (SiMD): it is an integral part of a physical device and governs its operation [3], as in the case of the firmware of a pacemaker or the control module of an infusion pump. *Standalone* software is instead termed **Software as a Medical Device** (SaMD), a term coined by the International Medical Device Regulators Forum [4]: it runs on general-purpose computing platforms, such as a server or a mobile device, and its medical intended purpose is independent of the platform on which it is installed. An application that analyses computed tomography images in order to flag the presence of suspicious lesions is one example. Figure 1.1 summarises the relationships between the categories introduced here.

The distinction has substantial regulatory consequences. For a SiMD, conformity assessment concerns the device as a whole; for a SaMD, the software is itself the product under assessment, and must therefore demonstrate its own clinical safety and performance.

1.1.3 AI-Based SaMD

Within SaMD, a subcategory has progressively established itself which constitutes the object of the present work: systems that employ artificial intelligence and

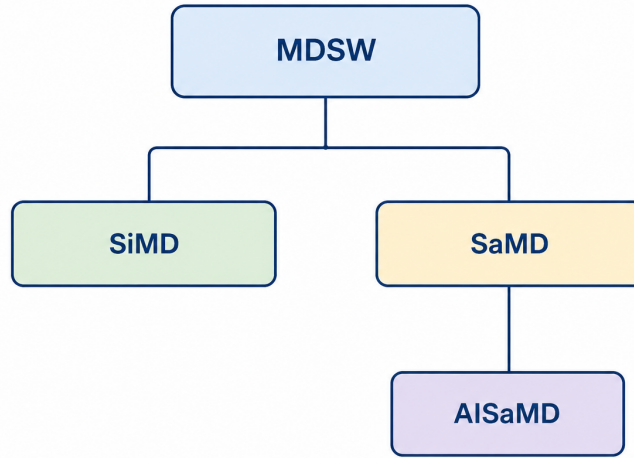


Figure 1.1: Relationships between the categories of medical software: SaMD as a subset of MDSW, distinct from SiMD, and AI-based systems as a further specialisation of it.

machine learning techniques to perform the clinical function. Their specific characteristics have motivated the adoption of a dedicated designation, *AI as a Medical Device* (AIaMD) [3].

Such systems present distinctive problems, which stem from their complexity and from the iterative, data-driven nature of the development process [3]. In a conventional system, behaviour is determined by explicit rules written by a developer, and verification can concentrate on the code and on its adherence to the requirements. In a system based on machine learning, behaviour emerges instead from the data used for training: the code describes the learning procedure, not the learned function. To this is added a limited operational transparency, whereby the reasons for a specific decision or recommendation are not immediately accessible either to clinicians or to patients.

A further distinction, relevant from a regulatory standpoint, separates *locked* models, whose parameters remain fixed after training, from *continuous learning* (adaptive) models, which automatically update their internal parameters as new data are acquired [3]. The possibility that already certified software may autonomously modify its own behaviour after being placed on the market remains an open problem for regulators.

1.2 The Regulatory Framework for Medical Devices in the European Union

Placing a Software as a Medical Device on the European market requires demonstrating conformity with an articulated regulatory framework. That framework defines the processes, the evidence and the documentation that the manufacturer must produce in order to demonstrate that the device meets those requirements throughout its entire life cycle.

1.2.1 Risk Classification under the MDR

The MDR adopts a risk-based regulatory approach, according to which the obligations imposed on the manufacturer are proportionate to the potential impact that a medical device may have on the health of the patient. As the risk associated with the device increases, so do the level of control required, the complexity of the conformity assessment procedure and the amount of documentary evidence needed to demonstrate compliance with the regulatory requirements [1, 3].

In order to apply this principle, the regulation divides medical devices into four risk classes, identified as I, IIa, IIb and III. Classification depends primarily on the intended purpose of the device, on the degree of invasiveness, on the duration of use and on the possible clinical consequences of a malfunction [1]. The four classes, reported in Table 1.1, represent increasing levels of risk.

Class	Risk level
I	Low
IIa	Medium-low
IIb	Medium-high
III	High

Table 1.1: Risk classes laid down by the MDR. Adapted from [3].

The risk class is not a merely descriptive classification: it determines the entire regulatory pathway that the device will have to follow before being placed on the market. In particular, it establishes the manner in which the manufacturer must demonstrate the conformity of the product and the level of verification required of the competent authorities.

Consequences for conformity assessment. For devices belonging to class I, the manufacturer may generally attest conformity on its own by means of a declaration of conformity, without the involvement of third parties. For all higher classes, the intervention of a **notified body** is instead required, tasked with assessing the technical documentation and the quality management system before the CE marking is issued. As the risk class increases, so does the depth of the assessment, as summarised in Table 1.2 [3].

Class	Conformity assessment
I	Self-declaration by the manufacturer, without third-party involvement
IIa	Notified body: quality system and technical documentation of at least one device per category
IIb	Notified body: quality system and technical documentation of at least one device per generic group
III	Notified body: quality system and technical documentation of each device

Table 1.2: Degree of control required in conformity assessment as a function of the class of the device. Adapted from [3].

From the standpoint of the present work, this aspect is of particular relevance, since the involvement of a notified body makes the technical documentation the principal instrument through which the manufacturer demonstrates the conformity of the device. The same requirement will return in Section 1.3, as it constitutes one of the conditions that determine the classification of an artificial intelligence system as *high-risk*.

The classification of medical software. Among the classification criteria laid down by the regulation, one is of particular importance for the present work, as it governs software specifically. It establishes that software intended to provide information used to take diagnostic or therapeutic decisions belongs at least to class IIa. The classification is raised to class IIb where such decisions may cause a serious deterioration of a person’s state of health or require surgical intervention, and it reaches class III in cases where they may cause death or an irreversible deterioration of health.

The same criterion assigns class IIa to software intended for monitoring physiological processes, raising it to class IIb when the monitoring concerns vital parameters whose variations may result in immediate danger to the patient. All other software without a specific medical purpose falls instead into class I [1, 3]. The resulting classification is summarised in Table 1.3.

Intended purpose of the software	Class
Information used to take diagnostic or therapeutic decisions	IIa
with potential impact of serious deterioration of health or surgical intervention	IIb
with potential impact of death or irreversible deterioration	III
Monitoring of physiological processes	IIa
monitoring of vital parameters entailing immediate danger	IIb
All other software	I

Table 1.3: Classification of medical software according to Rule 11 of the MDR. Adapted from [3].

This criterion has had a significant impact on the sector. Whereas in the past numerous software products were classified in class I, the introduction of the MDR has brought about a general raising of the risk class of Software as a Medical Device, with the result that most systems intended for diagnostic or therapeutic support fall at least into class IIa. Consequently, such devices are normally subject to assessment by a notified body and to the preparation of particularly extensive technical documentation.

The technical documentation. While the class determines the level of control required, documentary obligations are laid down by the regulation across the board. The manufacturer must demonstrate compliance with the general safety and performance requirements, prepare complete technical documentation for the device, and maintain documentation relating to post-market surveillance [1].

Taken together, these documents constitute the evidence through which the manufacturer demonstrates that the device meets the requirements throughout its entire life cycle. Conformity therefore depends not only on the quality of the software developed, but also on the ability to document its design, development,

validation and monitoring completely and verifiably.

1.2.2 The Relevant Technical Standards

The regulation establishes what the manufacturer must demonstrate, but does not address how the demonstration is to be carried out. That task is entrusted to a set of technical standards, developed by international standardisation bodies and recognised at European level, which translate regulatory requirements into operational processes. Compliance with them is not formally mandatory, but in practice it constitutes the ordinary route through which conformity is demonstrated and recognised.

Three of them are of particular relevance for the development of medical software, and they address three complementary aspects of the same problem.

The first aspect concerns the organisation that produces the device. **ISO 13485** [5] requires the manufacturer to adopt a quality management system with formalised and verifiable procedures for the entire production process, with particular attention to document control: who drafts documents, who approves them, how they are reviewed and archived. From this perspective, the quality of the product follows from the discipline of the process that generates it.

The second aspect concerns risk. **ISO 14971** [6] describes a cyclical process that starts from the identification of hazards, estimates their probability and severity, defines control measures and subsequently verifies their effectiveness. The process is not exhausted once the device is placed on the market, but continues through the collection of information arising from the actual use of the device.

The third aspect concerns software specifically. The two preceding standards, while essential, provide no guidance on software engineering practices; this is supplied by **IEC 62304** [7], which defines the activities to be carried out in each phase of development and the documentation that must result from them. Its organising criterion is a three-level safety classification, assigned according to the severity of the harm that a software malfunction could cause: it ranges from the level at which the software cannot contribute to any unacceptable risk, up to the level at which a defect in it may cause serious injury or death [3]. As the level rises, so do the rigour required of the development processes and the extent of the documentation to be produced.

1.3 The European Regulation on Artificial Intelligence

The framework described so far was conceived for devices whose behaviour is determined by design. When the device incorporates artificial intelligence components, as observed in Section 1.1.3, this premise no longer holds: the behaviour of the system emerges from the data used to train it, and verification of the code alone is no longer sufficient to demonstrate its safety.

It is this gap that Regulation (EU) 2024/1689 addresses, commonly known as the **AI Act** and published in July 2024 [8]. It is the first comprehensive regulatory framework dedicated to artificial intelligence systems, and it adopts a structure analogous to the one already encountered: obligations are proportionate to the risk the system entails. Some uses are prohibited outright; others, considered high-risk, are permitted only subject to stringent requirements; others still are subject only to transparency obligations or to no specific requirement at all.

1.3.1 AI-Based Medical Software as a High-Risk System

The regulation does not qualify artificial intelligence systems as high-risk on the basis of the technology employed, but on the basis of the context in which they operate. One of the routes laid down for such qualification concerns systems that constitute, or are incorporated into, products already governed by other European safety legislation, provided that those products undergo a third-party conformity assessment [8].

Both conditions occur in the case of an AI-based Software as a Medical Device. The MDR is indeed among the pieces of legislation referred to by the regulation; and, as seen in Section 1.2.1, software intended to support diagnostic or therapeutic decisions falls into a class that requires the intervention of a notified body.

A fundamental consequence follows: **the AI Act does not replace the MDR, but is superimposed on it**. Those who develop AI-based medical software are required to satisfy both bodies of obligations simultaneously, and in documentary terms this means that the required artefacts add up rather than replace one another.

It should be added that the obligations relating to this category of systems will become fully applicable only from August 2027, by virtue of a transitional regime.

The subject addressed in this thesis therefore concerns a deadline that is imminent but not yet operative, which increases the interest in tools that help manufacturers prepare for it.

1.3.2 Documentary Obligations

The AI Act likewise makes documentation the instrument through which conformity is demonstrated. Those who place a high-risk system on the market must prepare its technical documentation *before* commercialisation, and keep it up to date throughout the life cycle of the system [8]. The minimum content required is extensive, and ranges from the description of the system to the choices made during its development, from risk management to subsequent monitoring.

Among these contents there is one that distinguishes the AI Act from what has been seen so far: the documentation of the *data*. The provider must describe the datasets used to train, validate and test the model: where they come from, how they were obtained and selected, how they are organised, what size and characteristics they have. The provider must also account for the checks carried out to identify unsuitable sources or possible biases.

This is a requirement of a different nature from those encountered previously. The MDR requires the product and the process that produced it to be documented; the AI Act requires that the material from which the behaviour of the system emerged be documented as well. The reason is the one already noted: in a system learned from data, the data are part of the specification.

1.3.3 Data Quality and Governance

The document on which the case study developed in this thesis focuses derives from the provisions that establish which properties the data must possess and which practices must accompany their constitution [8]. They apply to the datasets used for the training, validation and testing of every high-risk system that makes use of machine learning techniques.

The requirements are of two kinds.

The first concerns **the way in which the data were constituted**. The provider must account for the design choices adopted, for the origin of the data and,

in the case of personal data, for the purpose for which they were originally collected; for the preparation operations to which they were subjected; for the assumptions formulated as to what the data are supposed to represent; for the assessment carried out on their availability and suitability; and finally for the examination performed to identify possible biases, together with the measures adopted to mitigate them.

The second concerns **the quality of the resulting datasets**. They must be relevant to the purpose of the system, sufficiently representative and, as far as possible, complete and free of errors; they must have appropriate statistical properties with respect to the persons for whom the system is intended; and they must take into account the geographical, behavioural or functional setting in which the system will be used.

Considered as a whole, these requirements exhibit a characteristic that constitutes the starting point of the present work. Some of them concern *facts* about the dataset (its structure, its composition, its origin, the applicable licences) which are in principle observable in project artefacts. Others concern instead *judgements* formulated by those who developed the system: the assumptions made, the outcome of the bias assessment, the suitability of the data for the intended purpose. The latter cannot be extracted from any file, since they reside in the knowledge of the developers and find expression, where they find it at all, in discursive documentation.

1.4 Technical Documentation as a Software Artefact

The preceding sections have outlined a framework in which the conformity of AI-based medical software is demonstrated through an extensive body of documentation, required by two regulations, whose obligations are cumulative. It remains to consider what this entails from the standpoint of software engineering.

The obligations described share three properties. They are first of all **mandatory**: documentation is not an optional by-product of development, and its incompleteness prevents the product from being placed on the market irrespective of the quality of the software that has been built. They are moreover **living**: both

regulations require them to be kept up to date throughout the life cycle of the product, so that every substantial modification to the system entails a corresponding documentary update. They are finally **anchored to the product**: the content required describes properties of the software and of the data which are, to a varying extent, observable in project artefacts such as the source code, the version control history, the configuration files and the existing technical documentation.

It is from the combination of these three properties that the problem addressed in this thesis originates. Regulatory documentation is drafted and updated manually, whereas the artefacts from which its content derives evolve with every modification of the code. The consequence is a systematic misalignment between what the software is and what the documentation states: a form of technical debt which, in the regulatory domain, entails not merely maintenance costs but a risk of non-compliance.

Chapter 2

Language Models, Retrieval, and Agents

The previous chapter has shown that the conformity of AI-based medical software relies on a substantial amount of documentation that must be maintained throughout the life cycle of the product. Since part of this documentation describes artefacts that evolve together with the software, its production and maintenance are natural candidates for automation.

This chapter introduces the technical concepts used in the proposed framework. It first discusses Large Language Models and the problem of hallucination, then introduces Retrieval-Augmented Generation as a way to ground generated content in external evidence. Corrective RAG is subsequently considered as a mechanism for evaluating the quality of the retrieved evidence and supporting abstention. The final part of the chapter introduces agentic systems and state-graph orchestration, which provide the execution model adopted in the remainder of the thesis.

2.1 Large Language Models

Large Language Models (LLMs) are neural language models based on the transformer architecture [9], trained on large collections of text and able to generate natural language from an input instruction. Their development has made it possible to automate tasks that require the production or transformation of textual content, including the drafting of structured technical documentation.

LLMs are generally trained in a self-supervised setting, where the objective is to predict the next token in a sequence [10]. As a consequence, the model learns statistical regularities in language rather than an explicit collection of verified facts. Large-scale models can also perform tasks described directly through natural-language instructions, often without additional task-specific training [10]. This property makes them suitable for applications in which a model receives contextual information and is asked to produce a textual description of it.

Their use, however, introduces the problem of **hallucination**: the generation of content that is linguistically plausible but not supported by the available information or by verifiable facts [11]. This behaviour is particularly relevant when the information required to answer a prompt is incomplete, since the model may still produce a plausible continuation instead of signalling the absence of sufficient evidence.

The literature distinguishes between *factuality* and *faithfulness* hallucinations [11]. The former concerns statements that contradict verifiable facts, whereas the latter occurs when generated content is inconsistent with the context supplied to the model. Faithfulness is the more relevant notion for this work. If a model receives evidence extracted from a software repository, the resulting description should remain supported by that evidence rather than introduce additional information.

A related issue is the ability to abstain. Models tend to favour complete answers even when the available context is insufficient [11]. In regulatory documentation, this behaviour is undesirable: an unsupported statement may appear valid to the reader, whereas an explicit absence of information can be identified and addressed.

For this reason, the use of LLMs in the framework requires two forms of control. Generated content must be grounded in evidence that can be traced back to the project, and the system must be able to avoid generation when that evidence is insufficient. The first problem is addressed through retrieval, while the second motivates the corrective mechanisms discussed later in this chapter.

2.2 Retrieval-Augmented Generation

A language model cannot directly access the contents of a private software repository, nor can it reliably provide the provenance of information generated from its

parametric knowledge. *Retrieval-Augmented Generation* (RAG) addresses this limitation by supplying the model with information retrieved from an external source at inference time [12].

In a typical RAG pipeline, the available documents are divided into smaller fragments, or *chunks*. Each fragment is represented through an embedding and stored in a retrieval index. When a query is received, the system identifies the fragments that are most similar to it and includes them in the context provided to the language model. Generation is therefore conditioned on information retrieved specifically for the current request.

This mechanism is useful for the problem considered in this thesis for two reasons. First, it reduces the dependence of the model on its parametric knowledge by providing the content needed for the requested description. Second, the retrieved fragments have a known origin, which makes it possible to associate generated content with the documents from which the supporting information was obtained.

Retrieval alone, however, does not guarantee that the selected fragments actually answer the request. A retrieval system ranks documents according to a similarity criterion, but the highest-ranked result may still be irrelevant or only loosely related to the information required. This is especially important when the corpus does not contain an answer: the retrieval phase may still return the closest available fragments.

The opposite problem can also occur. Relevant information may be present in the corpus but expressed with terminology that makes it difficult to retrieve for a given query. In either case, the generation phase receives evidence whose suitability has not yet been established.

A further control is therefore required between retrieval and generation: the retrieved evidence must be assessed before it is used to populate the document.

2.3 Corrective RAG

Corrective Retrieval-Augmented Generation (CRAG) introduces an explicit evaluation step between retrieval and generation [13]. The central idea is that retrieved documents should not be treated as valid evidence solely because they have been returned by the retrieval system.

CRAg employs a retrieval evaluator that estimates whether the retrieved content is relevant to the request [13]. Depending on the result of this evaluation, the system may use the retrieved material, reject it or apply additional processing before generation. The original approach also includes a decompose-then-recompose procedure that isolates the informative portions of retrieved documents and removes irrelevant content [13].

When local evidence is insufficient, the original CRAg formulation can extend the search to external web sources [13]. This behaviour is not suitable for the framework developed in this thesis. Regulatory documentation generated from a project should describe that project and should remain traceable to its artefacts. Information retrieved from external sources could be useful for general knowledge, but it cannot replace evidence that is absent from the repository.

For this reason, evidence evaluation is used here to support a different behaviour: **abstention**. If the available fragments do not provide sufficient support for a field, the system does not attempt to complete it from external knowledge. The absence of support is instead recorded explicitly. This mechanism is described in detail in Chapter 5.

2.4 LLM-Based Agentic Systems

RAG assumes that the retrieval operation to be performed is already known. The generation of a regulatory document is more heterogeneous: different fields may require different sources and different extraction strategies. Some information can be obtained deterministically, some must be searched in textual artefacts, and some may not be present in the repository at all.

An orchestration mechanism is therefore needed to select the appropriate operation for each part of the document. LLM-based agentic systems provide this capability by allowing a model to choose among a predefined set of actions according to the current state of the task.

2.4.1 Reasoning and Acting

ReAct is one of the reference formulations for combining language-model reasoning with actions performed on an external environment [14]. Rather than producing

the final answer immediately, the model can select an action, observe its result and use that observation to determine the next step.

Actions are exposed to the model as *tools*. A tool is an external function with a defined interface that performs a specific operation and returns its result to the model. The language model therefore decides which operation should be invoked, while the execution itself remains implemented in ordinary code.

This separation is useful in the proposed framework because not every operation should be performed by a generative model. Tasks such as reading metadata, inspecting the Git history or determining whether a file exists can be delegated to deterministic functions, while the model is used only where interpretation or orchestration is required.

2.4.2 State-Graph Orchestration

Allowing an agent to select tools does not by itself define the complete execution process. In a regulated setting, it is also useful to constrain which transitions are possible and to retain the intermediate state produced by each operation.

A state graph provides such a structure. Each node represents an operation, while the edges define the transitions that may occur between nodes. A shared state stores the information produced during execution and makes it available to subsequent components.

This representation has several advantages for the framework developed in this thesis. The set of admissible transitions is explicit, intermediate results can be inspected, and the sequence of operations can be reconstructed after execution. The model therefore retains some freedom in selecting actions, but only within a workflow whose boundaries are defined by the architecture.

These properties are particularly important for regulatory documentation. The final document should not only contain the required information; it should also be possible to determine where that information originated and which operations were used to obtain it. State-graph orchestration provides the execution structure on which the framework presented in the following chapters is built.

Chapter 3

Existing Approaches

The previous chapters introduced the regulatory problem addressed in this thesis and the technical mechanisms that may support its automation. This chapter places the proposed work within the existing literature, starting from software life-cycle automation and progressively moving towards regulatory documentation and controlled use of language models.

Section 3.1 considers DevOps and MLOps as the engineering context in which project artefacts and metadata are continuously produced. Section 3.2 discusses approaches that extend this automation to compliance activities. Section 3.3 focuses on structured documentation for machine learning systems and on recent work addressing the AI Act. Sections 3.4 and 3.5 examine mechanisms for controlling agentic systems and retrieval-based generation. Finally, Section 3.6 describes the previous work from which this thesis directly develops.

3.1 Life-Cycle Automation: from DevOps to MLOps

The automation of the software life cycle provides the engineering context for this work. If regulatory documentation is expected to remain aligned with a continuously evolving product, the development pipeline is a natural place in which to maintain part of that documentation.

Machine learning systems introduce engineering concerns that extend beyond source code. Data dependencies, configuration, models and training pipelines contribute substantially to the complexity of the system and to its technical debt [15].

This has led to the consolidation of MLOps practices, which treat the management of data, models and deployment pipelines as first-class engineering activities [16, 17].

These practices also leave persistent traces. Experiment tracking, data versioning and model management record metadata about the development process and the artefacts it produces [18]. Such information is mainly collected for reproducibility, collaboration and maintenance, but it can also provide evidence about how an ML system was developed.

The literature on MLOps therefore establishes an important premise for this thesis: a project repository already contains structured information that may be useful for documentation. However, the purpose of these systems is engineering rather than regulatory compliance. They do not generally determine whether the collected metadata satisfy specific legal obligations, nor do they produce documentary artefacts intended for external assessment.

3.2 RegOps and Compliance as Code

A related line of work extends development automation to regulatory compliance. The underlying idea is that compliance should be maintained throughout the life cycle rather than reconstructed only before an audit. This appears in the notions of *continuous compliance* and *living traceability*, first explored in regulated industrial environments and later applied to medical software [19, 20].

The relationship between DevOps and medical software regulation has also been studied by mapping the requirements of IEC 62304 onto continuous delivery practices. This analysis shows both the constraints introduced by regulation and the activities that can be supported through automation [21].

The term *RegOps* has been used to describe the systematic integration of regulatory requirements into delivery pipelines, in particular through the alignment of IEC 62304 and IEC 82304-1 with software development activities [22]. Related work has explored the use of GitHub to generate traceability information during ordinary development [23], the introduction of compliance checks into agile workflows [24], and the use of pull requests and model cards as design-control and audit mechanisms for machine learning components [25].

These approaches are close to the objective of this thesis because they treat compliance as something that can emerge from normal development activities. Their focus, however, remains primarily on verification, traceability and process control. Documentary content that cannot be obtained from mechanically readable meta-data is still generally written by a human operator.

3.3 Technical Documentation for the AI Act

Structured documentation for machine learning systems predates the AI Act. Model cards were introduced to describe models in terms of intended use, performance and limitations [26], while datasheets provide a corresponding structure for datasets, including their motivation, composition, collection, preprocessing, use and maintenance [27].

Recent work has begun to examine how these documentation practices relate to the obligations introduced by the AI Act. One study evaluates the ability of AI-based tools to draft and assess technical documentation for open-source high-risk systems, comparing their results with expert legal judgement [28]. Its starting point, however, is documentation that already exists: the system evaluates or rewrites it rather than deriving its content directly from project artefacts.

The template adopted in this thesis is provided by TechOps [29]. The approach proposes three open-source documentation templates, respectively covering datasets, models and deployed applications. Each field is associated with the corresponding regulatory requirement, thereby providing an explicit mapping between the document structure and the AI Act. The templates were validated through application to real cases and feedback from intended users.

Figure 3.1 shows the mapping between the regulatory requirements and the three templates. This thesis uses the data template, which covers the data-governance requirements of Article 10 together with related elements of the technical documentation, including metadata, standards, cybersecurity, risk management and monitoring.

The TechOps templates should be regarded as a research interpretation of the

AI Act Technical Documentation Requirements	TechOps Data Documentation Template	TechOps Model Documentation Template	TechOps Application Template
1. General Description of the AI System (I) (a) Intended purpose and version (II) (b) System interaction with software and hardware (III) (c) Software Versions and Updates (IV) (d) Deployment format (built-in software, download, APIs) (V) (e) Hardware requirements description (VI) (f) Visuals explaining product design and integration (VII) (g) User-interface description (VIII) (h) Instructions for the deployer (IX) 2. Detailed description of the elements and development of the system (X) (a) Development methods (XI) (b) Design specifications (XII) (c) System architecture (XIII) (d) Data requirements and datasheet (XIV) (e) Human-oversight measures (XV) (f) Pre-determined changes and solutions for compliance (XVI) (g) Validation, testing and performance metrics, logs (XVII) (h) Cybersecurity measures (XVIII) 3. Monitoring, functioning and control (XIX) 4. Performance metrics (XX) 5. Risk-management system (XXI) 6. Lifecycle changes (XXII) 7. Standards applied (XXIII) 8. EU declaration of conformity (XXIV) 9. Post-market monitoring (XXV)	1. Dataset Overview (I) (XIV) (a) Dataset Description (II) (b) Version and status (II, V) (c) Relevant links (V) (d) Developers (e) Owners (f) Instructions and user interface description (VIII, IX) 2. Data Versioning (IV) 3. Known Usages (XII) (a) Model(s) (b) Application(s) 4. Dataset Characteristics (XIV) 5. Data Origins and Source (X, XI) 6. Provenance (Collection Method(s) and Source Description) (XIV) 7. Data pre-processing (XI, XIV) (a) Data Cleaning (b) Data Transformation (c) Feature Engineering (d) Data Augmentation 8. Data Annotation (XV) 9. Dataset distribution (XIV, XXII) 10. Access, Retention, Deletion (XIX) 11. Data Risk Assessment (XVIII, XXI) 12. Cybersecurity Measures (Storage, Transfer, Processing) (XVIII) 13. Documentation Metadata (XXII) 14. Standards applied (XXIII) 15. Data Monitoring (XV)	1. Model Overview (I, II, IV) (a) Model Type (I) (b) Model Description (I, II) (c) Relevant links (V) (d) Developers (e) Owners (f) Version Details and Artefacts 2. Intended and Known Usage (In and Out of Scope Use) (VII, IX) 3. Model Architecture (X, XI, XII, XIII) (a) Datasheet (XIV) (b) Model Training (c) Model Validation (XVII) 4. Model Testing and Evaluation (XIX, XX, XVII) (a) Accuracy, precision, recall, F1 score for classification, - MSE, RMSE, MAE for regression. (b) ROC Curve and AOC 5. Model Bias and Fairness Analysis (XV, XXI) (a) Bias detection methods used (b) Mitigation Measures (XIX) (c) Retraining (XXII) (d) Post-Processing (XV) 6. Model Interpretability and Explainability (XV) 7. Documentation Metadata 8. Standards Applied (XXIII)	1. General Information (I) (a) Intended Purpose (II) (b) Status (c) Relevant links (V) (d) Developers (e) Owners 2. Intended/Known Usage (VII, IX, XII) 2. AI Act Risk Classification (XXI) 3. Data Documentation (XIV) 4. Model Documentation (XII, XIII) 5. Deployment (a) Infrastructure and Environment Details (III, V, VI) 6. Integration with External Systems (III, V) 7. Deployment Plan (XIX) (a) Risk Management System (XXI) (b) Risk Mitigation Measures (XX) 8. Testing and Validation (XVII) 9. Human Oversight (XV) 10. Monitoring (XIX, XV) 11. Incident Management (XV) 12. Standards applied (XXIII) 13. EU declaration of conformity

Figure 3.1: Mapping between the technical documentation requirements of the AI Act (numbered I to XXIV) and the fields of the three TechOps templates. Reproduced from Lucaj et al. [29], Figure 2. This work employs the data template only.

regulation rather than as official European Commission documentation. Their relevance lies in the explicit mapping between each field and the corresponding regulatory provision. They define what should be documented, but do not specify where each piece of information should be obtained, who should provide it, or how missing information should be handled. These questions motivate the field model introduced in Chapter 4.

3.4 LLM Agents and Behavioural Control

Section 2.4 introduced agentic systems as a mechanism for choosing among different actions during execution. For the problem considered here, the complementary issue is how that behaviour can be constrained.

One form of control is structural. Representing execution as a state graph makes the admissible transitions explicit and allows intermediate states to be inspected [30]. A second form concerns the evaluation of the model’s own behaviour. This may be implemented through self-reflection mechanisms, as in Self-RAG [31], or delegated to a separate component, as in Corrective RAG [13].

A further control is obtained by separating decision from execution. Operations that can be performed deterministically are implemented in ordinary code, while the language model is used to decide which operation should be selected or to interpret information that cannot be handled mechanically.

These mechanisms are relevant in a regulatory setting because the execution process must remain inspectable. It should be possible to reconstruct not only the final result, but also the evidence used and the sequence of operations that produced it.

3.5 RAG in Regulated Domains

Retrieval-augmented generation has also been applied in domains where unsupported statements may have significant consequences. In these settings, retrieval is used both to reduce reliance on parametric knowledge and to associate generated content with identifiable sources.

The setting considered in this thesis differs from many conventional RAG applications. In typical knowledge-retrieval scenarios, the corpus is designed to contain the information that the system is expected to retrieve. A software repository is different: some of the information required by a regulatory document may simply never have been recorded.

For this reason, retrieval failure is not necessarily an exceptional condition. It may instead correctly indicate that the repository does not contain sufficient evidence for a given field. A system designed for regulatory documentation must therefore be able to distinguish between information that has not been retrieved and information that does not exist in the available project artefacts.

3.6 The Starting Point

The immediate starting point for this thesis is the author’s previous work on MLOps-driven documentation [32]. That work investigated whether regulatory documentation could be generated as part of the MLOps pipeline rather than prepared manually after development.

The proposed system integrated documentation generation into a CI/CD workflow. When a pull request was opened, a Python orchestrator executed through GitHub Actions collected project metadata by inspecting the source tree, dependency files and Git history, and by querying package metadata for external libraries. The extracted information was normalised into JSON and used to populate Jinja2 templates that were stored in the repository together with the software history.

The generated artefacts were the *Software List* and the *SOUP List*, which record software components and components of unknown provenance relevant to IEC 62304. The system was evaluated on the CT-COVID repository and extracted metadata for more than thirty external packages, covering most of the fields defined by the templates.

The present thesis retains the same general idea: documentation should be derived from the development process, the repository should act as a primary source of evidence, and missing information should be made visible rather than silently replaced.

The previous approach, however, has three limitations.

First, extraction is entirely deterministic. It works well for structured metadata but cannot recover information expressed in prose across README files, notebooks or technical reports.

Second, the distinction between automatic and manual fields is fixed in advance. A field is classified as non-automatable by the template itself, rather than according to the evidence actually available in a specific repository. The system therefore cannot determine at runtime whether sufficient information exists for that field.

Third, the previous work addresses documentation derived from the MDR and IEC 62304, but not the additional requirements introduced by Regulation 2024/1689 for high-risk AI systems, including Articles 10 and 11 and Annex IV.

These limitations define the starting point for the framework developed in the following chapters.

Chapter 4

Data Documentation: Fields and Strategies

The preceding chapter has identified the space in which this work is situated: none of the systems examined generates the technical documentation required by the AI Act from the evidence contained in a repository, distinguishing the nature of the information required by each field and declaring the fields for which no evidence exists.

Before building a system that does so, it is necessary to establish what such a system should produce. This chapter answers three questions, in order: what the document requires, where each piece of information resides, and what to do when it resides nowhere. The answers constitute the *field model*, that is, the data structure that describes the document and that, in Chapter 5, will constrain the behaviour of the architecture. It is the only chapter of this thesis that does not speak of software components: it deals with the document, not with the program that writes it.

4.1 From the Template to the Field Model

The starting point is a template, that is, the translation of the articles of the regulation into a structure of sections and fields, each annotated with the regulatory reference that justifies it. The template adopted is the one relating to data proposed by TechOps [29] and discussed in Section 3.3. The resulting document, the *Data Documentation*, gathers the obligations on training, validation and testing data

laid down in Article 10 together with the corresponding items of the technical documentation required by Article 11 and Annex IV.

A template of this kind fulfils one precise function and leaves three others uncovered. It establishes *what* must appear in the document, and in this it is irreplaceable: without it the set of fields would be arbitrary. It does not establish, however, where the information is to come from, nor who is to supply it, nor above all what should happen when it does not exist.

In current practice the three gaps are filled implicitly by the same person: whoever drafts the document knows where to look, knows that it falls to them, and when they find nothing they write something all the same. An automated system has no such implicit knowledge. And the cost of this absence is not symmetric.

A field left empty is indeed a visible defect, which whoever verifies the document detects immediately. A field filled with plausible but ungrounded content is instead an invisible defect, which survives verification precisely because it has the appearance of an answer. Regulatory documentation is a textual genre in which fluency is no index of quality: a language model can produce a credible paragraph on how a dataset was annotated even when no trace of such annotation exists in the project.

The field model answers this problem by associating with each field of the template, before execution, three pieces of information that the template does not contain: the nature of the information required, the strategy by which it can be obtained, and the set of outcomes the system is authorised to produce for that field. The first two are the subject of the two following sections, the third of Section 4.5.

4.2 Where the Information Resides

The repository of a machine learning project is not a homogeneous source. Examining the fields of the *Data Documentation*, one observes that the information required is distributed across three distinct regimes, which differ not in content but in the manner in which the content is inscribed in the artefacts.

The first regime is that of the **observable fact**. The information exists in structured form, produced by a tool and not by a person: the licence is the `LICENSE` file, the developers are the authors of the commits, the size of the data is recorded

in the pointers of the versioning system, the stages of preprocessing are the stages declared in the configuration file of the pipeline. Here no interpretive ambiguity exists: either the artefact is there, in which case the value is determined, or it is not. These are the fields for which a language model is not merely superfluous but harmful, because it would introduce a possibility of error into an operation that has none.

The second regime concerns **discursive information**. In this case, the required content is available in textual artefacts rather than in structured fields. Examples include a README describing data provenance, a technical report explaining how training and test sets were defined, a notebook documenting applied transformations, or source-code comments describing how missing values were handled.

These fields cannot be populated through direct extraction. Instead, the relevant information must first be located within the available text, assessed for relevance, and then reformulated into the structure required by the document. This is the main setting in which language models are used in the framework. Their use, however, also introduces the possibility of interpreting the available evidence incorrectly or adding information that is not explicitly supported by the repository.

The third regime is that of **judgement**. Here the information does not reside in the repository as a matter of principle, not because of a documentary gap: the conformity assessment, the names of the reviewers, the technical standards the organisation declares it applies, the policies for data retention and deletion are not properties of the code, but decisions of a responsible party. No improvement of the retrieval system will ever extract them, because there is nothing to extract. The only correct behaviour is to declare that filling the field falls to a person.

4.3 The Three Strategies

To the three regimes correspond three filling strategies. The assignment is made once, at the stage of modelling the document, and holds for every execution, this being a property of the field.

The `tool` strategy applies to observable facts. The field is filled by a deterministic extractor which queries the artefact and returns, together with the value, the paths of the files from which it comes; no language model intervenes in the

formation of the value. The outcome is constrained: either the value is determined, or the artefact is absent and the field is marked as not observable. This second declaration is informative as well, because the absence of a licence file is itself a fact relevant to whoever verifies the document.

The **retrieval** strategy is, in the other hand, applied to discursive information and it is used to fill fields through four steps. Retrieval selects, within the closed corpus of the textual artefacts of the project, the fragments relevant to the field. Grading assesses their sufficiency. Generation produces the text using exclusively the fragments judged relevant by the previous component, and citing them. Verification checks that every generated sentence is supported by those fragments. If retrieval finds nothing, the field is not documentable. If it finds insufficient material, the system abstains. If verification detects an unsupported statement, the text is rejected rather than corrected.

The **human** strategy applies, finally, to fields of judgement. The field is left unfilled: the system requests explicit human intervention and never completes it automatically. This is not an abdication but the correct behaviour, and its correctness is measurable: a system that automatically filled in the declaration of conformity would produce a worse document, not a better one.

4.4 Application to the Data Documentation

It remains to apply the criterion to the actual fields of the document.

The TechOps template organises the *Data Documentation* into fifteen sections. Assigning a strategy has, however, required a finer granularity than that of the template, because fields that the template treats as a unit turn out, in practice, to be of different natures: the status date of a dataset is read from the version control history, whereas the status it is in must be argued in prose, even though the template gathers them under a single item. Decomposing cases such as this, together with the addition of three items required by Article 10 but not made explicit in the template (the validation method, the sampling criterion and the declaration of conformity), brings the document to forty fields distributed over eighteen sections. Table 4.2, at the end of this section, reports them in full.

The distribution across the three strategies is the one shown in Table 4.1.

Table 4.1: Distribution of the forty fields across the three strategies.

Strategy	Fields	Share
tool	18	45 %
retrieval	18	45 %
human	4	10 %
Total	40	100 %

In this template, the two automatable strategies are well balanced. Almost half of the document can be completed without using a language model, eliminating the risk of hallucination for those fields. This reflects the maturity of the MLOps practices discussed in Section 3.1: those practices produce, for engineering purposes, metadata that turn out to have demonstrative value as well. An almost equal share requires instead the mediation of a model, and it is there that the entire risk of the system is concentrated. The remaining share is irreducible by construction.

Equally instructive is the reading by section. The sections that describe the *identity* of the dataset (overview, versioning, distribution and licensing, documentation metadata) are predominantly deterministic. Those that describe its *construction* (provenance, preprocessing, annotation, sampling, risk assessment) are largely discursive.

From this asymmetry follows a direct consequence for the evaluation of the system, taken up in section 5.14: an aggregate metric over the forty fields overestimates its quality, since eighteen of them are correct by construction. Metrics must therefore also be reported disaggregated by strategy.

Table 4.2: The forty fields of the Data Documentation, grouped by section, with their assigned strategy and regulatory reference. A dash indicates a field that derives from the structure of the template rather than from a specific regulatory requirement.

Field	Strategy	Normative link
Overview		
Dataset name	TOOL	Annex IV 1
Dataset Description	RETRIEVAL	Art. 11; Annex IV 2(d)
Status Date	TOOL	—

continued on next page

continued from previous page

Field	Strategy	Regulatory links
Status	TOOL	—
Relevant Links	TOOL	—
Developers	TOOL	—
Owner	TOOL	—
Deployer instructions of use	RETRIEVAL	Art. 13
Version Details	TOOL	—
Data Versioning		
Data Version Control Tools	TOOL	Art. 11 2(d)
Maintenance of Metadata and Schema	RETRIEVAL	—
Versioning		
Known Usages		
Model(s)	TOOL	—
Application(s)	RETRIEVAL	—
Dataset Characteristics		
Data Types	TOOL	—
Size	TOOL	—
Primary Use Case(s)	RETRIEVAL	—
Data Origin and Source		
Source(s)	RETRIEVAL	Art. 10 2(b)
Provenance		
Collection Method(s)	RETRIEVAL	Art. 10 2(b)
Dates of Collection	RETRIEVAL	—
Collection Cadence	RETRIEVAL	—
Data Pre-Processing		
Pre-processing pipeline (observed)	TOOL	Art. 10 2(c)
Data Cleaning	RETRIEVAL	Art. 10 2(c)
Data Transformation	RETRIEVAL	Art. 10 2(c)
Feature Engineering	RETRIEVAL	—
Data Augmentation	RETRIEVAL	—
Data Annotation and Labeling		

continued on next page

continued from previous page

Field	Strategy	Regulatory links
Annotation process	RETRIEVAL	Art. 10 2(c)
Validation Types		
Validation method(s)	TOOL	—
Sampling Methods		
Sampling / split	RETRIEVAL	Art. 10 2(f)
Dataset Distribution and Licensing		
Distribution	TOOL	—
License	TOOL	—
Access, Retention, and Deletion		
Access	RETRIEVAL	—
Retention & Deletion	HUMAN	—
Data Risk Assessment		
Bias / risk examination	RETRIEVAL	Art. 10 2(f)(g)
Cybersecurity Measures		
Security measures	RETRIEVAL	—
Documentation Metadata		
Documentation authors	TOOL	—
Document version	TOOL	—
Reviewers	HUMAN	—
Standards Applied		
Standards	HUMAN	—
Data Monitoring		
Monitoring	TOOL	—
EU Declaration of Conformity		
Declaration	HUMAN	—

4.5 The Admissible Outcomes

Having established what the document requires and by which strategy each field can be filled, the third question remains: what the system is authorised to declare

when filling does not succeed.

A document in which every field is either full or empty does not allow situations to be distinguished that, from the point of view of whoever verifies it, are profoundly different. The model therefore provides for five outcomes, reported in Table 4.3.

Table 4.3: Admissible outcomes for a field and their meaning.

Outcome	Meaning
filled	the field has been filled and, if generated, verified
partial	the evidence covers only in part what the field requires
not observable	the artefact from which the value would be read is absent from the repository
abstained	the evidence was sought and judged insufficient
requires human input	the field falls within the competence of a person

The three outcomes other than *filled* and *partial* are not gradations of one and the same failure. *Not observable* concerns a fact and signals a gap in the project; *abstained* concerns a text and signals a gap in the internal documentation; *requires human input* signals no gap at all, but a competence. Whoever receives the document acts differently in each of the three cases: this is why the distinction belongs to the model of the document, and not to the implementation. In the case study of Chapter 5 the first outcome never occurs, because every artefact the extractors read is present in the repository; on a project without a licence file or without data versioning it would.

For such outcomes to be verifiable, every filled field must moreover carry its own provenance: the paths of the artefacts from which the value derives, for deterministic fields, and the references to the fragments cited, for generated ones. A filled field whose source is unknown is, for the purposes of demonstrating conformity, equivalent to an empty field.

Chapter 5

An Agentic Framework for Data Documentation

The field model establishes what the system must produce and under which constraints. This chapter describes the system that realises it: LLMAutodoc, a framework that takes the address of a repository as input and returns the data documentation document together with the trace of how each field was obtained. This chapter presents the architecture which the system is based on, the specifications linked to each required field and the description of each individual component which contribute to the final result.

5.1 The Architecture

Figure 5.1 show the architecture of LLMAutodoc system, organised into three nested area scopes.

The outermost scope is the **Oversight**. The whole of the LLMAutodoc architecture is overseen by a person: the one who triggers a run, by opening a pull request on the repository to be documented, and who receives the resulting regulatory documentation. In a strictly regulated setting this involvement is not optional. The document must return to that person to be checked and completed wherever a value is missing or unclear, and the responsibility for what it states remains theirs.

Within the Oversight lies the **RegOps** scope, which conducts the entire path from the repository URL to the regulatory artefact. It opens with the Setup, which

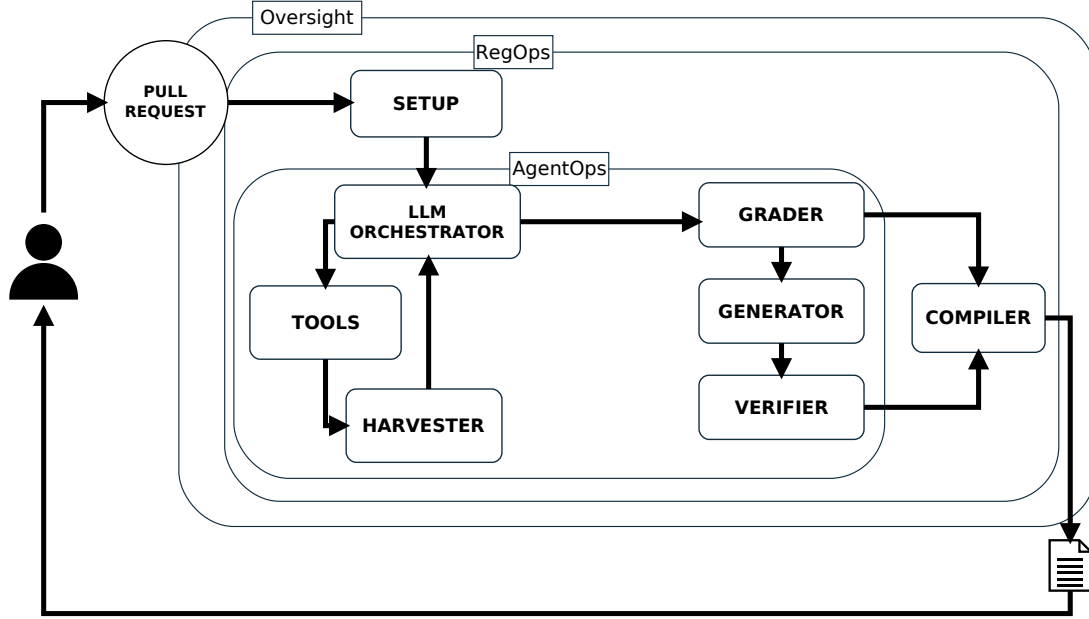


Figure 5.1: The architecture of LLMAutodoc.

clones the repository and initialises the forty fields from the specification, and it closes with the Compiler, which renders the document and writes the audit trail. Both the Setup and the Compiler node are deterministic.

The innermost scope is **AgentOps**. It includes the components in which a language model is involved, together with the mechanisms used to constrain their behaviour.

The central node is the LLM Orchestrator, which manages the execution flow and decides which tool should be used for each field according to the field specification. The available tools include deterministic extractors and the retrieval tool. The Tools node executes the selected operations, while the Harvester collects their results and stores them in the shared state.

The remaining three nodes operate on retrieval-based fields. The Grader evaluates the retrieved fragments and determines whether they contain sufficient evidence. The Generator uses the accepted fragments to produce the text for each field, while the Verifier performs a final check to determine whether the generated content is supported by the available evidence.

The workflow follows two execution patterns.

The first is a *loop*. The Orchestrator reads the field specification and decides

which tools to invoke. The Tools node executes the selected operations, and the Harvester stores their results in the shared state before returning control to the Orchestrator. The same process is repeated until the Orchestrator determines that no additional tool calls are required.

The second flow is a *chain*. Once collection ends, the Grader, the Generator, the Verifier and the Compiler are executed once, in this order, each iterating internally over retrieval fields, whose content must be checked before being added to the document.

One edge departs from this order and deserves attention. From the Grader a transition leads directly to the Compiler, bypassing generation and verification. The system follows it when no discursive field has survived grading, in which case there is no text to write. The document that results is composed of the deterministic fields and of the abstentions, each carrying the reason it was recorded.

5.2 The Field Specification

The field model of Chapter 4 is not a description that the components carry in their own code. It is a separate artefact, declarative and inspectable, which the components read. Table 5.1 shows its structure.

Table 5.1: The attributes of a field specification.

Attribute	Declared for	Content
id	every field	key of the field in the shared state and in the template
section	every field	section of the document the field belongs to
label	every field	name of the field as it appears in the document
strategy	every field	tool retrieval human
aiact	every field	article or annex of the regulation the field descends from
tool	tool fields	name of the deterministic extractor to run
query	retrieval fields	query submitted to the retrieval tool
keywords	retrieval fields	indicative terms, used by the lexical heuristic of the LLM-free baseline
guidance	retrieval fields	one sentence stating what the field must contain; read by the Grader and by the Generator

The **strategy** states which of the three regimes the field belongs to, and therefore which components are allowed to touch it. The **aiact** attribute records the article or annex from which the field descends, so that every entry can be traced back to the obligation that requires it. The **guidance** states what the field must contain, in one sentence. The remaining attributes are the operational consequence of the strategy: the name of the extractor for a deterministic field, and the query and the indicative terms for a discursive one. Table 5.2 show two entries of the specification.

Table 5.2: Two entries of the specification: an observable field and a discursive one.

	datasetName	datasetDescription
section	Overview	Overview
label	Dataset name	Dataset Description
strategy	tool	retrieval
aiact	Annex IV 1	Art. 11; Annex IV 2(d)
tool	dataset_name	—
query	—	dataset description content topic what the data contains purpose of the dataset images scans records
keywords	—	dataset, data, images, records, samples, collected
guidance	—	Summary (max 200 words) of what the dataset contains, the task it serves, its source and motivation

One design choice is worth highlighting. The guidance consists of a single sentence shared by both the Grader and the Generator, ensuring that the two components rely on the same interpretation of the expected content. In addition, the specification is kept separate from the implementation: supporting a new document type only requires defining a new specification file and the corresponding extractors, without changing to the overall architecture.

5.3 Initialization and Shared State

The execution starts by acquiring the repository. If the argument points to a local directory, it is used as is; otherwise the repository is cloned into a temporary folder. The specification is then instantiated in the shared state, a single data structure

that spans the entire graph and is shared by all components. This becomes the only structure the various components can use to communicate with each other. The shared state collects the execution configuration, the repository path, a record for each of the forty fields, the trace of what each component has accomplished, and the orchestrator conversation. For each field, in addition to the ID, the document section to which it belongs, and the strategy defined to obtain it, it records the current state of the field, the sources from which the value derives, an indication of which component covered it, and a note explaining the condition. For these last three fields, the information reported is not incidental. Since, in the first case, the sources that make the content verifiable are reported. Finally, before releasing the shared state, all fields with the `human` strategy are set as requiring human intervention, with the reason already recorded in the note, and no component will reconsider them. The system cannot obtain these fields; a person must fill them. The remaining thirty-six fields enter the next phase in a waiting state.

5.4 Evidence Gathering

Immediately following initialization and the creation of the shared state, the process moves to the phase of gathering information needed to populate the fields specified in the documentation. Control of the entire process is handed over to the orchestrator, which has two families of tools at its disposal to retrieve the necessary information, mirroring the strategies defined in the field model. The first is a **deterministic** approach comprising 18 extractors—one for each observable field. The second involves a single **retrieval** tool that queries the project’s text corpus, using only documents or evidence found within it. The orchestrator selects the appropriate tool based on the specific field required. It manages the collection process by issuing method calls—grouping several within a single turn—and determines when the collection phase is complete. Each tool returns a one-line summary to the model and stores the full result where the **Harvester** node collects it, ensuring that the conversation context does not expand in proportion to the volume of evidence gathered. However, coverage does not rely solely on that decision. When the orchestrator concludes its operations, or when a limit on the number of turns is reached, a deterministic mechanism executes the designated tool for any field still

lacking evidence and logs its intervention. Consequently, the trace distinguishes between fields covered by the orchestrator and those addressed by this mechanism; this renders the reliability of the agentic layer an observable quantity rather than a mere assumption.

5.4.1 Observation Tools

Observation tools are deterministic Python functions, one for each field in the specification associated with the ‘tool’ strategy. Each function receives the repository path and returns an ‘Evidence’ structure containing the observed value, the sources from which it was extracted, and a note if none is available. The ‘EXTRACTORS’ dictionary handles the mapping between the name of the extractor declared in the specification and the Python function. Extractors work on artefacts typically found in a machine learning project, such as Git history, configuration files, DVC artefacts, license files, dataset structure, and source code. This allows them to extract information such as the project version, repository activity, data versioning tools, license, dataset composition, and some pipeline characteristics. Executing an extractor can only produce two outcomes: ‘filled’, when the value is observable, or ‘not observable’, when the required artifact is not available. The model therefore does not contribute to the generation of these fields. When simple inference is required, as in the case of ‘dataset status’, the applied rule and the underlying data are recorded along with the result. From the orchestrator’s perspective, each extractor is exposed as a separate tool. The repository path is provided by the shared state, not the model. Only a summary of the outcome is returned to the orchestrator, while the complete result is retained for subsequent document construction. This separation limits context growth and keeps the observation phase separate from the orchestration phase.

5.4.2 The Retrieval Tool

In addition to deterministic extraction tools, the orchestrator has a single tool for fields with a `retrieval` strategy: `retrieve evidence`. This tool searches for information within the repository and returns the most relevant text fragments for the field in question.

The corpus is built from project files. These include Markdown and text files, PDF reports, Markdown cells from Jupyter notebooks, docstrings and comments in Python files, as well as some configuration files such as `params.yaml`, `dvc.yaml`, and `config.yaml`. The content is divided into fragments of limited size, attempting to keep consecutive paragraphs together while separating parts belonging to different sections.

The search is performed with BM25. The query is not generated by the model, but derives from the `query` and `keywords` attributes defined in the field specification. This choice makes the retrieval reproducible and independent of model decisions. For each field, up to `top k` relevant fragments are returned, while those without terms in common with the query are excluded.

The search is also limited to the analyseddd repository. The system does not query external sources or use the web to fill in missing information.

Unlike deterministic extractors, `retrieve evidence` does not directly produce the final value of the field. Instead, it returns the collected evidence with the `unverified` status. Their interpretation and transformation into the final content of the document occurs in subsequent phases.

5.5 Evidence Grading and Abstention

At the end of the collection phase, for each field using the `retrieval` strategy, the state contains only the retrieved fragments, which must be checked before actually use in the regulatory context. Therefore, it is necessary to verify the quality of the obtained fragments and organize them once they have been deemed valid.

This assessment is entrusted to an evaluation model, distinct from the generator. The model receives the field label, the corresponding section, the `guidance`, and the numbered fragments. Its task is to determine whether the content of the fragments provides information sufficiently relevant to the field being analyseddd. All this is done without relying on external knowledge. This distinction is important: a fragment that is semantically close to the topic is not necessarily sufficient to document the project.

The evaluator’s output includes a verdict—CORRECT, AMBIGUOUS, or INCORRECT—along with the indices of the fragments deemed relevant and a brief justification. A CORRECT verdict indicates that at least one fragment directly addresses the field; AMBIGUOUS signals partial or indirect information; INCORRECT indicates that the available evidence is insufficient. The selected indices also define the set of evidence that may be used during the generation phase.

The model’s output is subsequently validated. A positive verdict lacking relevant fragments, as well as an unrecognized verdict, is reclassified as INCORRECT; invalid indices are discarded. If the retrieval process yields no fragments, the field is rejected without querying the model.

In the event of a negative outcome, the field is assigned the **abstained** status. The field is left with a null value, and a note is added explaining the reason for the failure to complete it. Furthermore, the grader tracks additional information about the corpus, such as the total number of fragments and the files from which they originate. This ensures a record is always maintained of the sources used to determine the final outcome, whether positive or negative.

Finally, the assessment result influences the pipeline’s workflow. If no discourse field passes the evaluation, the generation and verification stages are skipped, and execution proceeds directly to document compilation.

The grader can also be disabled via configuration. In this case, all retrieved fragments are forwarded to the generation stage without any corrective checks. This mode serves as the baseline for experimentally comparing the effect of the assessment mechanism, which is discussed in Section 5.12.

5.6 Controlled Generation and Faithfulness Verification

Fields that pass the grading phase enter the generation stage. This stage applies only to fields associated with the `retrieval` strategy. Deterministic fields have already obtained their value directly from the corresponding extractor, while fields requiring human judgement are excluded from automatic processing from initialization.

The Generator does not receive the complete set of fragments returned by retrieval. Instead, it is given only those fragments whose indices were selected by the Grader as relevant to the field.

For each field, the Generator receives its label, the **guidance** defined in the field specification, and the selected evidence fragments. The model is instructed to use exclusively the information contained in those fragments, without completing missing details through external knowledge or inference. Each factual statement must be associated with a numerical reference to the supporting fragment. When part of the information requested by the field is absent, the generated text must explicitly state that it is not documented in the repository.

The state assigned to the resulting field depends on the previous grading decision. Evidence judged **CORRECT** produces a **filled** field, whereas evidence judged **AMBIGUOUS** produces a **partial** field.

Grounded generation alone, however, does not guarantee that the produced text remains faithful to the supplied evidence. A language model may still introduce an unsupported fact while paraphrasing or combining the retrieved fragments. For this reason, generated fields are passed to a separate Verifier before compilation.

The Verifier receives the generated text together with the same evidence fragments used for its production. Its task is different from that of the Grader. The Grader evaluates whether the retrieved evidence is suitable for answering the field; the Verifier evaluates whether the answer actually written is supported by that evidence. It therefore performs a sentence-level faithfulness check rather than another relevance judgement.

For each factual statement, the Verifier determines whether the information it contains can be traced to the supplied fragments.

A negative verification result prevents the generated value from reaching the final document. The field is converted to **abstained**, its generated value is discarded, and the unsupported content is recorded in the field note. Consequently, generation is not treated as the final authority over the document.

5.7 Compilation, Provenance and Audit Trail

The Compiler is the final node of the pipeline. It receives the shared state after verification, or directly after grading when no retrieval field contains sufficient evidence to proceed with generation. Unlike the previous components, it does not perform extraction, retrieval, judgement, or generation. Its role is to transform the final state of the execution into the two artefacts produced by LLMAutodoc: the regulatory documentation and its audit trail.

The documentation is rendered through a Jinja2 template [33]. Before rendering, the Compiler groups the fields according to the sections defined in the field specification and prepares the values required by the template, including the repository identifier, the execution date, the routing mode, the corpus statistics and the number of fields associated with each final status.

For each field, the rendered document preserves the distinction introduced by the field model. A field can appear as **filled**, **partial**, **abstained** or **human_required**. For all these cases, it reports for each field the fragments used to provide the final values and their position into the repository.

The second output of the Compiler is `audit.json`. This file serializes the information needed to reconstruct how each field reached its final state. For every field, it records the strategy, status, value, sources, the component that covered it and, for retrieval fields, the Grader verdict and the result of the faithfulness verification. At run level, the audit also stores the routing mode, the execution configuration, corpus statistics, the number of Orchestrator turns and the node-by-node execution trace. The API key is explicitly excluded from the serialized configuration.

The audit additionally records whether a field was covered by the Orchestrator, by the deterministic coverage fallback, by the declarative routing mode, or directly by the field specification in the case of human fields. This distinction is important because it makes the behaviour of the agentic layer observable.

The same principle applies to abstention. When the Grader rejects the retrieved evidence, or when the Verifier rejects a generated statement, the resulting null value is accompanied by the evidence, decision and rationale that led to it. The system therefore preserves information about unsuccessful paths rather than discarding them. This allows an auditor to inspect not only why a field was filled, but also why another field was deliberately left without a value.

For this reason, the documentation and the audit trail should be considered complementary outputs of the same execution. The former is the regulatory artefact intended for human consumption; the latter provides the provenance required to verify how that artefact was produced. Without the audit trail, a generated field could be read but not independently traced to the evidence and decisions that produced it. Conversely, the audit trail without the rendered document would expose the execution without providing the regulatory artefact itself.

5.8 Implementation

The framework is implemented in Python, with a modular structure that follows the architecture described in the previous sections. Each main architectural component is implemented as a separate software module, while the execution flow is coordinated through a shared state.

The orchestration layer is implemented using **LangGraph**, which provides the state-graph abstraction used to represent the workflow. Nodes correspond to the main processing stages of the framework, while conditional edges determine the admissible transitions between them. This allows the execution path to remain explicit and makes the intermediate state available to each component.

Interaction with the language models and the tool interface is implemented through **LangChain**. The Orchestrator uses the same tool abstraction described in Section 5.4 to invoke deterministic extractors and the retrieval tool. The language-model components are accessed through the Anthropic API. Separate model instances are used for generation-oriented tasks and for judgement-oriented tasks: the Orchestrator and Generator use the main model, while the Grader and Verifier use a lighter model configured for evidence assessment.

Retrieval is implemented with BM25 through the `rank_bm25` library. The choice avoids the need for an embedding model and keeps the retrieval stage deterministic for a fixed corpus and query. Text contained in PDF reports is extracted with `pypdf`, while the remaining corpus is built directly from the textual artefacts present in the repository.

The final document is produced through a Jinja2 template [33]. The Compiler receives the values accumulated in the shared state and renders the corresponding

document, while a separate JSON file stores the complete audit information associated with the run. This includes the status of each field, its evidence, the outcome of grading and verification, and the execution trace.

Run parameters are kept outside the implementation in a YAML configuration file. Model selection, routing strategy, retrieval parameters and the activation of the Grader and Verifier can therefore be changed without modifying the code. Sensitive information, such as API credentials, is not stored in the configuration and is instead read from environment variables.

5.9 Case Study

To understand the proposed framework final behaviour, it was evaluated on a real case study. The case study considered for the application uses CT-COVID¹ [34], a public repository for COVID-19 detection from computed tomography images. This repository was selected because it contains a set of artefacts relevant to the presented framework—such as Git history, documentation files, license information, DVC artefacts, a technical report in PDF format, and other items useful for the subsequent experimentation.

In particular, the repository provides both structured and unstructured sources of information, allowing the framework to exercise the different extraction strategies described in the previous chapter. At the same time, not all the information required by the final Data Documentation is explicitly available, making the repository suitable for evaluating how the system behaves when evidence is incomplete or absent. This combination makes CT-COVID a representative case for assessing both automatic field population and abstention mechanisms.

5.10 Research Questions

To evaluate the experiment, we chose to try to answer three research questions:

RQ1 Is the system able to produce the Data Documentation document required by the regulation on AI-based medical software, comparing it with a manually

¹<https://github.com/se4ai2122-cs-uniba/CT-COVID>

created ground truth?

RQ2 Is the developed system dependent on the family of models chosen?

RQ3 Is the Orchestrator able to operate, obtaining good results, even when the tool to be used is not indicated for each field of the final document?

RQ1 evaluates the overall ability of the system to produce the Data Documentation defined in Chapter 4. The generated fields are compared against a manually constructed ground truth in order to measure how much of the required information is correctly identified, populated, or left unresolved when evidence is missing.

RQ2 examines the dependence of the system on the model family used in the pipeline. The reference configuration uses Claude models, while additional runs assign equivalent roles to Gemini models. The goal is to observe whether the main results remain stable when the underlying model family changes.

RQ3 focuses on the agentic behaviour of the Orchestrator. In the default configuration, each field specification already indicates the admissible strategy. By removing this information, the experiment tests whether the Orchestrator can still select the appropriate tool or strategy from the field description alone.

5.11 Ground Truth

The ground truth was constructed by considering the forty fields specified in the document subject to evaluation. For each field, one of three possible values was manually assigned: *present*, *partial*, or *absent*.

A field is classified as *present* when the repository contains the required information in a sufficiently explicit manner; as *partial* when the information is available only in part or indirectly; and as *absent* when there is no adequate evidence to satisfy the field’s requirement.

For fields classified as *present* or *partial*, the ground truth also specifies the terms expected in a correct answer and, where necessary, those that should not appear. The annotation was performed manually by a single annotator.

As shown in Table 5.3, all eighteen fields associated with the `tool` strategy are observable in the repository and can therefore be handled via deterministic extractors. The four fields classified as `human` must not be automatically populated.

Table 5.3: Ground truth distribution by strategy associated with the fields.

Strategy	Present	Partial	Absent	Total
tool	18	0	0	18
retrieval	7	3	8	18
human	0	0	4	4
Total	25	3	12	40

The most important part of the evaluation will focus about the eighteen **retrieval** fields. Among these, seven are classified as *present*, three as *partial*, and eight as *absent*. The latter are particularly useful for assessing the system’s behaviour in cases where the requested information is unavailable, making it necessary to distinguish between generation and abstention.

5.12 Experimental Settings

The following Table 5.4 summarises all the evaluated configurations.

Table 5.4: Experimental configurations used in the ablation study.

Configuration	Grader	Verifier	Strategy hints	LLM	Purpose
full	yes	yes	yes	yes	complete pipeline
no-verify	yes	no	yes	yes	remove faithfulness verification
no-grade	no	yes	yes	yes	remove evidence grading
infer-strategy	yes	yes	no	yes	infer the strategy automatically
dry-run	heuristic	no	—	no	deterministic baseline

The **full** configuration corresponds to the architecture described in Chapter 5.

In **no-verify**, generated retrieval-based fields are not passed to the Verifier. The comparison with **full** therefore isolates the effect of the final faithfulness check.

In **no-grade**, retrieved chunks are passed directly to the Generator. This configuration approximates a conventional retrieve-and-generate pipeline in which no explicit decision is made about evidence sufficiency before generation.

In **infer-strategy**, the Orchestrator receives the field section and label but not its assigned strategy. Guidance is also omitted because it is only defined for retrieval fields and would indirectly reveal their strategy. The resulting tool selection is compared with the strategy declared in the specification.

Finally, **dry-run** removes language-model calls entirely. Deterministic extractors are executed normally, while retrieval evidence is accepted through a lexical heuristic based on the field keywords. Accepted chunks are concatenated rather than rewritten by a Generator. Since an LLM-based Orchestrator is not available in this configuration, routing is also declarative. The configuration should therefore be interpreted as a deterministic baseline rather than as an isolated ablation of the language model.

Each LLM-based configuration was executed twice. Results are reported as mean values, with standard deviation where it is non-zero. The reference configuration uses **claude-sonnet-4-6** for orchestration and generation and **claude-haiku-4-5** for grading and verification, with temperature set to zero.

In addition, Three runs were performed to compare model families. They use Gemini for selected roles while using the same framework’s workflow:

- Claude as the main model for orchestration, generation, and grading, with Gemini used only for verification;
- Gemini as the main model for orchestration, generation, and grading, with Claude used only for verification;
- a **no-grade** variant combining Claude for generation with Gemini for verification.

The Gemini model used in these runs is **gemini-3.5-flash-lite**. Since the models used across providers are not equivalent in capacity, this experiment evaluates robustness across model families rather than a controlled comparison of model quality.

5.13 Evaluation metrics

Evaluation is performed at the individual field level by comparing the system’s output with the corresponding reference label (*ground truth*). For each field, two aspects are considered: first, whether sufficient information is available in the repository to populate the field; and second, whether the system actually chooses to populate it or to abstain.

Table 5.5: Field-level evaluation results.

	Information present or partial	Information absent
System populates the field	True positive	False positive
System abstains	False negative	True negative

Analysis of the results for the four cases shown in Table 5.5:

A *true positive* occurs when the repository contains relevant information and the system populates the field. A *false negative* corresponds to a field that could have been populated based on available information but for which the system abstains. A field is considered a *false positive* when the system produces content even though the repository does not provide enough evidence to support that field. If the required information is missing and the system leaves the field unfilled, the outcome is instead classified as a *true negative*.

The evaluation uses four metrics.

Coverage describes how much of the information available in the repository is recovered by the system. The calculation considers only fields for which some evidence exists, either complete or partial, and checks how many of them are eventually populated. In this sense, higher coverage means that fewer usable pieces of information are missed during the process.

Fill precision looks at the populated fields from a different perspective. Rather than considering only whether a field was filled, it also checks the generated content. Each populated field is compared with the expected terms defined in the *ground truth*, and the metric reports the share of generated fields that contain those expected elements. It therefore provides an indication of whether the system is not only filling fields, but also producing content that matches what is expected for them. A high value indicates that when the system provides an answer, it tends to contain the expected information.

Abstention precision measures the reliability of the system’s decisions not to populate a field. It indicates the proportion of abstentions that actually occur in cases where the repository does not contain the requested information. High abstention precision therefore means that when the system decides not to populate a field, the choice is generally correct because the repository lacks sufficient useful information.

Finally, the *hallucination rate* measures the fraction of absent fields that the system incorrectly marks as filled; therefore, lower values are better. However, the term hallucination should be interpreted with some caution. In practice, the metric counts any false fill, meaning a value generated for a field that is absent in the ground truth.

From the generated texts, three types of false fill can be identified. First, the system may simply state that the requested information is not documented in the repository, without making any unsupported claim. Second, it may provide correctly cited information that is present in the repository but does not directly answer the target field, for example describing image resizing under data cleaning or attention layers under feature engineering. Third, it may generate statements that are not supported by the cited evidence, which corresponds to hallucination in the strict sense.

Since the metric does not distinguish between these cases, it should be interpreted as an upper bound on the actual hallucination rate. Section 5.14 further analyses how the false fills are distributed across the three categories for each configuration.

Taken together, these metrics allow for the distinction of two complementary system properties. Coverage measures how effectively available information is used, while the hallucination rate measures the system’s caution when such information is unavailable. Fill precision evaluates the quality of the fields actually produced, whereas abstention precision verifies whether decisions not to provide an answer are justified by the actual absence of information. This distinction is particularly important in the context at hand, as a system might achieve high coverage by populating many fields but do so at the cost of a higher number of omitted answers; conversely, a highly conservative system might reduce hallucinations but increase the amount of available information that is ignored.

The metrics are computed on the full set of forty fields and also on the eighteen *retrieval-based* fields alone. The discussion mainly refers to this second group, because these fields are the most useful for assessing the retrieval and generation process. By contrast, deterministic fields and fields requiring human input are mostly governed by predefined system behaviour and are therefore less informative for this part of the evaluation.

It is also necessary to consider the limitations of the evaluation procedure. First, fill precision is based on the presence of expected terms defined in the ground truth. While this provides a reproducible and easily verifiable criterion, it does not constitute a comprehensive semantic assessment of the generated text’s correctness. Second, none of the four metrics reads the generated text: they record whether a field was filled or left empty and whether the expected terms appear, not whether the text would be acceptable as documentation.

5.14 Final Results

Table 5.6 presents the results for the eighteen fields obtained using the retrieval methodology:

Table 5.6: Results for the 18 retrieval-based fields: 7 present, 3 partial, and 8 absent. Values are averaged over two runs; standard deviation is reported when non-zero.

Configuration	Coverage	Fill precision	Abstention precision	Hallucination rate
full	0.800	0.875	0.778	0.125
no-verify	0.800	0.875	0.778	0.125
infer-strategy	0.800	0.875	0.778	0.125
no-grade	0.900	0.778	0.250 ± 0.25	0.938 ± 0.06
dry-run	0.800	0.875	0.778	0.125

Table 5.7 reports the corresponding values for the complete set, also including the fields obtained using deterministic methodology

Table 5.7: Results for the complete set of 40 fields.

Configuration	Coverage	Fill precision	Abstention precision	Hallucination rate
full	0.929	0.962	0.846	0.083
no-verify	0.929	0.962	0.846	0.083
infer-strategy	0.929	0.962	0.846	0.083
no-grade	0.964	0.926	0.817 ± 0.01	0.625 ± 0.04
dry-run	0.929	0.962	0.846	0.083

full. The full configuration serves as the baseline for comparison with the other variants. Across the eighteen fields, the **retrieval** approach achieves a coverage of 0.800, a fill precision of 0.875, an abstention precision of 0.778, and a hallucination rate of 0.125. The two runs yield identical results—both at the aggregate level and for individual fields—indicating stable behaviour in the case study under consideration.

no-verify. Removing the Verifier results in no changes compared to the full configuration. Coverage, fill precision, abstention precision, and hallucination rate remain unchanged across both runs. A field-by-field comparison likewise reveals no differences.

This result indicates that, within the repository under consideration, none of the texts generated after passing the Grader are subsequently rejected by the Verifier. Consequently, the fidelity check does not alter the final document in this specific evaluation.

no-grade. Removing the Grader, conversely, produces the most significant change. Coverage increases from 0.800 to 0.900, but this rise is accompanied by a drop in fill precision—from 0.875 to 0.778—and, most notably, a surge in the hallucination rate from 0.125 to 0.938 ± 0.06 . Abstention precision also decreases sharply, dropping to 0.250 ± 0.25 . Increased coverage does not, therefore, equate to an improvement in the system. Without a preliminary assessment of the sufficiency of the evidence, almost all fields are sent to the generation phase, including those for which the repository lacks adequate information. The two runs do not produce exactly the same outcomes, and this variability is reflected in the standard deviation reported for the metrics.

A closer analysis of the false fills clarifies what the hallucination rate actually captures. In the two no-grade runs, seven and eight of the eight absent retrieval fields are filled. In both runs, five fields simply state that the requested information is not documented in the repository. The remaining false fills contain information that is supported by the cited evidence but answers a related, rather than the intended, field. No unsupported statement appears in either run, a result confirmed both by the Verifier and by manual inspection. The only false fill in the full configuration belongs to this second category and is therefore graded as ambiguous.

The hallucination rate of 0.938 should therefore be interpreted as a false-fill rate rather than as a measure of fabricated content. In this repository, no unsupported content reaches the final document in any configuration. The main effect of the Grader is instead on how missing evidence is represented: with the Grader, absent fields become explicit abstentions in the trace, while without it the same absence is expressed as text in the document. As a result, the metric overstates the difference in hallucination between configurations and does not fully capture the difference in auditability.

infer-strategy. When the Orchestrator does not receive an indication of the strategy to use from the specification, the final metrics remain identical to those of the full configuration. In both runs, the Orchestrator also selects the strategy prescribed by the specification for 37 out of the 40 fields.

All eighteen **tool** fields are correctly associated with their respective extractors, while all eighteen **retrieval** fields are directed toward evidence retrieval. The three errors involve fields classified as **human**, for which the Orchestrator nonetheless attempts to use retrieval. These attempts do not alter the final output, as the system prevents the use of a strategy not permitted by the specification. In this repository, the Orchestrator can distinguish observable fields from discursive ones without explicit guidance, but it does not identify fields for which no automated tool should be used. The fact that these three errors do not affect the final document is due to the constraints on the allowed strategies rather than to the Orchestrator itself.

dry-run. The deterministic baseline achieves the same aggregate values as the **full** configuration: 0.800 coverage, 0.875 fill precision, 0.778 abstention precision, and a 0.125 hallucination rate. However, identical metrics do not equate to identical outputs.

In the **dry-run** configuration, no natural language generation takes place. Tools are executed according to the declared strategy, and for **retrieval** fields, the snippets accepted by the lexical heuristic are returned directly. Consequently, qualitative analysis reveals outputs that may contain heterogeneous fragments, headers, author names, contact details, or other information extraneous to the content required by the field.

The fill precision metric primarily considers the presence of expected terms and can therefore yield a positive result even for text that would not be suitable for use as final documentation. The alignment between the `dry-run` and `full` results thus highlights a limitation of the chosen metric rather than demonstrating quality.

Overall, the comparison between the configurations indicates that the Grader is the component that has the most significant impact on the metrics under consideration. The Verifier does not alter the outputs when the Grader is active, while removing strategy indications does not change the final result, thanks to the constraints imposed by the system. The deterministic baseline yields comparable aggregate values but produces qualitatively different documents and does not allow for isolating the contribution of language generation from that of orchestration.

Qualitative results analysis. The metrics in Section 5.13 capture whether a field is filled and whether the expected terms are present, but they do not assess the quality of the generated text. A manual inspection adds two observations.

The `dry-run` and `full` configurations obtain identical metric values but produce very different outputs. In the `dry-run`, retrieval fields contain the raw chunks selected by the lexical heuristic, including unrelated headers, author names and contact details. In the `full` configuration, the same evidence is rewritten into a coherent paragraph that directly addresses the field.

Then, none of the fields counted by the hallucination rate contains fabricated content. Most false fills in the `no-grade` runs simply state that the requested information is not documented, while the others contain supported information that answers a related but different field. The single false fill in the `full` configuration is of this latter type.

Therefore, the rates in Table 5.6 should mainly be interpreted as measures of the fill/abstain decision rather than of text quality. A proper assessment of the latter would require a separate human evaluation.

In terms of execution time, a full run takes approximately 110 seconds. The Orchestrator employs two interaction rounds and invokes the necessary tools for the thirty-six automatically processable fields. The Grader runs eighteen times, while the Generator and Verifier are activated nine times each.

The `no-verify` configuration takes approximately 90 seconds. The `no-grade` configuration, conversely, takes about 180 seconds, as the absence of the Grader

allows a larger number of fields to proceed to the subsequent generation and verification stages. The `dry-run` baseline, which does not use language models, completes execution in approximately five seconds. Execution times for configurations using Gemini are not directly compared, as they include delays caused by the provider’s rate limits.

Answer to RQ1. On the CT-COVID repository, the full configuration is able to produce the Data Documentation while preserving the distinction between information that can be extracted automatically and information that should remain unresolved. All eighteen deterministic fields are populated with the expected values, whereas the four fields requiring human judgement are correctly left for manual input.

Among the eighteen retrieval-based fields, eight are populated and ten result in abstention. Seven of the populated fields contain the expected information, corresponding to a fill precision of 0.875. One unsupported field is nevertheless filled, giving a hallucination rate of 0.125. Two fields for which some information is available are missed, resulting in a coverage of 0.800, while abstention precision reaches 0.778. When all forty fields are considered, coverage is 0.929, fill precision 0.962, abstention precision 0.846, and the hallucination rate 0.083.

Overall, the system recovers most of the information that is available in the repository and, when it fails, it more often leaves a field unresolved than fills it with unsupported content. The two runs of the full configuration also produce the same field-level outcome. These results should still be interpreted within the limits of the case study, which is based on a single repository, one document template, one annotator, and a fill-precision metric that checks for expected terms rather than the overall quality of the generated text.

5.15 Comparison Across Model Families

The second part of the evaluation concerns the framework’s dependence on the specific model family employed. The reference configuration uses Claude models for all roles: `claude-sonnet-4-6` for orchestration and generation, and `claude-haiku-4-5` for grading and verification.

To determine whether the results depend significantly on this choice, three additional configurations were executed in which certain roles were assigned to Gemini models, while the remaining components were kept unchanged.

This comparison does not aim to establish which model family is superior. The Gemini model used is `gemini-3.5-flash-lite` for all roles, whereas in the reference configuration, generation and control are handled by different Claude models; thus, the models’ capabilities are not directly equivalent. Instead, the objective is to observe whether the pipeline’s overall behaviour remains stable when generation and control are assigned to different families, and whether a Verifier from a family other than the one that generated the text produces different decisions.

The Gemini-based configurations were executed once each, whereas the values for the `no-grade` configuration with the Claude Verifier represent the average of the two runs from the experimental campaign.

Table 5.8: Comparison across configurations based on different model families. The C/A/I column reports the number of *correct*, *ambiguous*, and *incorrect* judgments assigned by the Grader over the 18 discursive fields.

Configuration	Orch./Gen.	Grader	Verifier	C/A/I	Rejected	Fill Prec.	Halluc.
<code>full</code>	Claude	Claude	Claude	6/3/9	0/9	0.875	0.125
External verifier	Claude	Claude	Gemini	6/3/9	0/9	0.875	0.125
Symmetric	Gemini	Gemini	Claude	5/4/9	0/9	0.750	0.125
<code>no-grade</code>	Claude	—	Claude	—	1–2/18	0.778	0.938
<code>no-grade</code> , external verifier	Claude	—	Gemini	—	0/18	0.700	1.000
<code>no-grade</code> , all Gemini	Gemini	—	Gemini	—	0/18	0.700	1.000

Evidence grading. The Grader’s behaviour remains similar across the two families. In the reference configuration, Claude assigns six *correct* verdicts, three *ambiguous*, and nine *incorrect*; with Gemini, the figures become five, four, and nine, respectively.

The total number of rejected fields thus remains unchanged, although three fields receive a different verdict. `sampling` shifts from *correct* to *ambiguous*, `riskAssessment` from *incorrect* to *ambiguous*, while `annotation` moves from *ambiguous* to *incorrect*. In each of the three disputed fields, the available evidence is incomplete rather than entirely absent. A clear example is `riskAssessment`: the technical report discusses

class imbalance, but it does not provide a dedicated risk analysis. The disagreement between Claude and Gemini therefore seems to concern the threshold used to judge whether partial evidence is sufficient. Outside these three cases, the Graders agree on all remaining fifteen fields.

Fidelity check. Replacing Claude with Gemini only at the verification stage does not change the outcome. All nine texts accepted by the Grader are also accepted by Gemini. As a result, the final metrics are unchanged with respect to the full configuration, with fill precision at 0.875 and hallucination rate at 0.125.

A similar result is obtained when the model families are swapped in the opposite direction. Gemini handles orchestration, generation, and grading, while Claude is used for verification, and again all nine texts pass the final check. The main difference is found in fill precision, which falls to 0.750, whereas the hallucination rate remains unchanged at 0.125. In cases where the evidence has already passed the Grader’s check, using a Verifier from a different family does not alter the number of accepted texts or the observed hallucination rate. The most noticeable difference, however, concerns the fill precision in the Gemini-based configuration.

This result indicates that the model family can influence the content produced for the completed fields, even when the decision to complete or abstain remains similar. The difference thus primarily concerns the quality of the final wording rather than the pipeline’s general behaviour regarding the availability of evidence.

Orchestration. In the Gemini-based configuration, the role of Orchestrator is also assigned to the same model family used for generation. The model completes the processing of the thirty-six automatable fields in two turns—as in the Claude-based configuration—without requiring the activation of the framework’s fallback mechanism.

In the case study considered, the orchestration cycle therefore shows no significant differences between the two families.

behaviour without Grader. A more marked difference emerges when the Grader is removed. In the `no-grade` configuration using Claude for generation and Gemini as the Verifier, none of the eighteen generated texts are rejected. The same situation is observed in the `no-grade` configuration based entirely on Gemini, where

the Verifier again accepts all eighteen texts.

A different result is observed when Claude is used as Verifier, since one or two texts are rejected in each run. The corresponding **no-grade** configuration obtains a fill precision of 0.778. By comparison, both setups using Gemini for verification obtain 0.700, independently of the model used to generate the text.

This difference is also visible in the hallucination rate, which is 0.938 with Claude and rises to 1.000 in both configurations using Gemini as Verifier. The composition of the false fills is the same across Verifiers: with the Gemini Verifier the eight absent fields divide into five explicit statements of absence and three grounded but off-scope texts; with Gemini in every role, into six and two. In neither case does a fabricated statement reach the document. The difference in rejection counts between the two Verifiers therefore concerns texts that a manual reading would also accept as faithful, which points to a difference in strictness rather than in accuracy.

The Verifier’s responses were correctly interpreted in all cases; thus, the difference does not stem from parsing errors or unreadable responses. Rather, the observed behaviour reflects a difference in the model’s strictness when evaluating faithfulness.

The fact that the Gemini Verifier rejects no text—whether evaluating outputs generated by Claude or by Gemini—further suggests that, in this configuration, this permissive behaviour depends primarily on the model used for verification rather than on the model family that produced the text.

In this case study, the strictness of the Verifier appears to depend on the model family when no preliminary filtering by the Grader is applied.

Validity of responses. In all the reported configurations, the responses from both the Grader and the Verifier were recorded as correctly interpreted.

Answer to RQ2. In the case study considered, the complete configuration does not show a strong dependency on the model family employed. With the Grader in place, the overall behaviour remains stable across the tested configurations. The hallucination rate stays at 0.125, and every text that reaches the Verifier is accepted. Most differences between the two model families appear instead in the Grader judgments, especially for fields supported only by partial or indirect evidence.

The choice of model family still has an effect on the generated content. When Gemini is used for orchestration, generation, and grading, fill precision decreases from 0.875 to 0.750, while the hallucination rate remains unchanged.

Dependency on the model family becomes more evident when the Grader’s check is removed. Under this condition, the Claude Verifier rejects one or two texts per run, whereas the Gemini Verifier rejects none—whether evaluating texts generated by Claude or when the entire configuration is based on Gemini. In both of these latter cases, fill precision is 0.700 and the hallucination rate reaches 1.000.

In this case study, the framework is not fully independent of the model family. However, the results suggest that separating the roles, particularly through the Grader’s preliminary evidence check, reduces the influence of model choice on the final outcome.

Answer to RQ3. When the strategy is withheld from the Orchestrator, together with the guidance text that would reveal it, the document does not change: the metrics of infer-strategy are identical to those of full in both runs. Looking at the routing rather than at the outcome, the Orchestrator sends 37 of the 40 fields to the strategy declared in the specification. All eighteen tool fields go to their extractor and all eighteen retrieval fields go to the retrieval tool; the three misses are the human fields, for which the Orchestrator attempts a retrieval instead of leaving the field to a person. These attempts are blocked by the framework, which does not allow a strategy that the specification forbids, and the human fields end up unresolved as in full.

The Orchestrator can therefore tell, from the section and the label alone, whether a field is to be observed or to be searched for in the text, and it obtains the same result as when it is told. What it does not recognise on its own is a field that no tool should handle. In the case study this had no effect on the document, because the constraint on admissible strategies caught the three cases; without that constraint, the three human fields would have been treated as retrieval fields. Within the limits of this case study, the Orchestrator’s routing is adequate for choosing between tools, and the decision not to use any tool is better left to the specification. Whether the same holds on repositories with a less regular structure is not something these 40 fields can establish.

Conclusions and Future Work

In this thesis, we explored the possibility of generating technical documentation for AI-based medical software using evidence from a repository, while aiming to minimize the risk of the model producing unsupported content.

The core idea of this work was to decouple information gathering from its evaluation and the subsequent text generation. This approach allows for the explicit handling of cases where available information is insufficient, preventing a lack of evidence from automatically translating into a plausible-sounding answer.

Evaluation results indicate that this approach is particularly useful when the system must decide whether or not to populate a specific field. The Grader is important because it decides whether the available evidence is sufficient before any text is generated. Without this step, the final result depends more strongly on the model used for verification. This was particularly visible in the **no-grade** experiments, where the two Verifiers did not behave in the same way. At the same time, no configuration in this case study produced fabricated content. The main effect of the Grader was instead to turn missing evidence into an explicit abstention rather than into text in the final document.

The comparison between Claude and Gemini leads to a similar observation. When evidence is checked before generation, changing the model family has only a limited effect on the final outcome. Once this check is removed, the differences become easier to observe, especially at the verification stage.

The orchestration experiments provide a different result. In the case study, the Orchestrator was able to choose the expected tool for almost all fields, 37 out of 40, even when the specification did not explicitly provide the routing information. The final results were not affected by this added freedom. This holds for the repository considered; the three errors concerned human fields, and the safeguard that made

them harmless should be regarded as part of the result. Further experiments would be needed to understand how useful this capability is on less structured projects.

The main contribution of LLMAutodoc is therefore not simply that it can generate regulatory documentation. More importantly, it introduces controls around the generation process and keeps the produced content connected to the evidence from which it was derived. Each field is linked to its supporting evidence, a lack of information can be represented as an abstention, and the execution trace allows for the reconstruction of the path leading to the final result.

This aspect is particularly important in a regulatory context. An empty field accompanied by a justification can be more valuable than text that appears complete but lacks sufficient evidentiary support. The framework therefore seeks to prioritize traceability and prudence over mere document completeness.

However, the evaluation has certain limitations. The experiments were conducted on a single repository, CT-COVID, and on only one of the documents specified by the template. The repository is relatively small, well structured, and well documented, so the results should be interpreted as specific to this case study rather than as evidence of general behaviour. The ground truth was created by a single annotator, and some fields required interpretation. The metrics also capture specific aspects of the system, such as fill/abstain decisions and the presence of expected terms, rather than the overall quality of the generated text. Furthermore, the limited number of runs makes it impossible to assess model variability in a general sense.

A useful next step would be to test the framework on a broader set of repositories, including projects with different structures, domains, and levels of documentation quality. The ground truth could also be strengthened by involving more than one annotator, especially for fields where the available evidence can be interpreted in different ways.

Future work should also consider the remaining documents included in the regulatory template. This would make it possible to assess whether the same architecture can be applied beyond the data documentation considered in this thesis. The architecture was designed to be reusable, but this property has yet to be experimentally verified.

One possible development involves integrating the Model Context Protocol

(MCP), which would make it easier to expand the set of tools usable by the framework. In this way, the system could access new sources and services without requiring custom integrations for each individual tool, making the architecture more flexible and easier to extend.

Finally, the framework can also be viewed as a tool for identifying gaps in project documentation. Instances of abstention and missing evidence do not merely indicate what the system failed to populate; they also highlight information that is absent from the repository. This opens up the possibility of using LLMAutodoc during the development process, rather than solely when generating the final documentation.

Bibliography

- [1] Parlamento europeo e Consiglio dell'Unione europea. Regolamento (ue) 2017/745 relativo ai dispositivi medici. Gazzetta ufficiale dell'Unione europea, L 117, 2017. CELEX 32017R0745. <https://eur-lex.europa.eu/eli/reg/2017/745/oj>.
- [2] Medical Device Coordination Group. Mdcg 2019-11: Guidance on qualification and classification of software in regulation (eu) 2017/745 – mdr and regulation (eu) 2017/746 – ivdr. European Commission, 10 2019.
- [3] Philip S. Cosgriff and Matthew J. Memmott. *Writing In-House Medical Device Software in Compliance with EU, UK, and US Regulations*. CRC Press, Taylor & Francis Group, Boca Raton, FL, 1 edition, 2024. ISBN 9781032293486.
- [4] International Medical Device Regulators Forum. Software as a medical device (samd): Key definitions, 12 2013. IMDR-F/SaMD WG/N10FINAL:2013. <https://www.imdrf.org/documents/software-medical-device-samd-key-definitions>.
- [5] International Organization for Standardization. Iso 13485:2016 — medical devices — quality management systems — requirements for regulatory purposes, 2016. <https://www.iso.org/standard/59752.html>.
- [6] International Organization for Standardization. Iso 14971:2019 — medical devices — application of risk management to medical devices, 2019. <https://www.iso.org/standard/72704.html>.
- [7] International Electrotechnical Commission. Iec 62304:2006/amd1:2015 — medical device software — software life cycle processes, 2015. <https://webstore.iec.ch/publication/22790>.
- [8] Parlamento europeo e Consiglio dell'Unione europea. Regolamento (ue) 2024/1689 che stabilisce regole armonizzate sull'intelligenza artificiale.

- Gazzetta ufficiale dell’Unione europea, L, 2024/1689, 2024. CELEX 32024R1689. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>.
- [9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30:5998–6008, 2017. DOI: 10.48550/arXiv.1706.03762.
- [10] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020. DOI: 10.48550/arXiv.2005.14165.
- [11] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55, 2025. DOI: 10.1145/3703155.
- [12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020. DOI: 10.48550/arXiv.2005.11401.
- [13] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation. *arXiv preprint arXiv:2401.15884*, 2024. DOI: 10.48550/arXiv.2401.15884.
- [14] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. DOI: 10.48550/arXiv.2210.03629.
- [15] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, 28:2503–2511, 2015. <https://papers.nips.cc/paper/>

5656-hidden-technical-debt-in-machine-learning-systems.

- [16] Saleema Amershi, Andrew Begel, Christian Bird, Robert DeLine, Harald C. Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. Software engineering for machine learning: A case study. In *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 291–300. IEEE, 2019. DOI: 10.1109/ICSE-SEIP.2019.00042.
- [17] Dominik Kreuzberger, Niklas Köhl, and Sebastian Hirschl. Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11:31866–31879, 2023. DOI: 10.1109/ACCESS.2023.3262138.
- [18] Samuel Idowu, Daniel Strüder, and Thorsten Berger. Asset management in machine learning: State-of-research and state-of-practice. *ACM Computing Surveys*, 55(7):1–35, 2023. DOI: 10.1145/3543847.
- [19] Brian Fitzgerald, Klaas-Jan Stol, Ryan O’Sullivan, and Donal O’Brien. Scaling agile methods to regulated environments: An industry case study. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, pages 863–872. IEEE, 2013. DOI: 10.1109/ICSE.2013.6606635.
- [20] Tuomas Granlund, Vlad Stirbu, and Tommi Mikkonen. Medical software needs calm compliance. *IEEE Software*, 39(1):19–28, 2022. DOI: 10.1109/MS.2021.3117292.
- [21] Teemu Laukkarinen, Kati Kuusinen, and Tommi Mikkonen. DevOps in regulated software development: Case medical devices. In *Proceedings of the 39th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, pages 15–18. IEEE, 2017. DOI: 10.1109/ICSE-NIER.2017.20.
- [22] Henrik Toivakka, Tuomas Granlund, Timo Poranen, and Zheyang Zhang. Towards RegOps: A DevOps pipeline for medical device software. In *Product-Focused Software Process Improvement (PROFES 2021)*, volume 13126 of *Lecture Notes in Computer Science*, pages 290–306. Springer, 2021. DOI: 10.1007/978-3-030-91452-3_20.
- [23] Vlad Stirbu and Tommi Mikkonen. Introducing traceability in GitHub for medical software development. In *Product-Focused Software Process Improvement (PROFES 2021)*, volume 13126 of *Lecture Notes in Computer Science*, pages 152–164.

- Springer, 2021. DOI: 10.1007/978-3-030-91452-3_10.
- [24] Vlad Stirbu and Tommi Mikkonen. CompliancePal: A tool for supporting practical agile and regulatory-compliant development of medical software. In *Proceedings of the 17th International Conference on Software Architecture — Companion (ICSA-C)*, pages 151–158. IEEE, 2020. DOI: 10.1109/ICSA-C50368.2020.00035.
- [25] Vlad Stirbu, Tuomas Granlund, and Tommi Mikkonen. Continuous design control for machine learning in certified medical systems. *Software Quality Journal*, 31(2):307–333, 2023. DOI: 10.1007/s11219-022-09601-5.
- [26] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, pages 220–229. ACM, 2019. DOI: 10.1145/3287560.3287596.
- [27] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. DOI: 10.1145/3458723.
- [28] Francesco Sovrano, Emmie Hine, Stefano Anzolut, and Alberto Bacchelli. Simplifying software compliance: AI technologies in drafting technical documentation for the AI act. *Empirical Software Engineering*, 30:91, 2025. DOI: 10.1007/s10664-025-10645-x.
- [29] Laura Lucaj, Alex Loosley, Håkan Jonsson, Urs Gasser, and Patrick van der Smagt. TechOps: Technical documentation templates for the AI act. In *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society (AIES)*, volume 8, pages 1647–1660, 2025. DOI: 10.1609/aies.v8i2.36663.
- [30] LangChain Inc. LangGraph: Documentation. <https://docs.langchain.com/oss/python/langgraph/overview>. Consultato nel 2026.
- [31] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024. <https://iclr.cc/virtual/2024/oral/19736>.
- [32] Fabrizio Rosmarino, Giulio Mallardi, Luigi Quaranta, and Filippo Lanubile. Mlops-driven automation of regulatory documentation for ai-based medical software. 10 2025. DOI: 10.13140/RG.2.2.22830.06726.

- [33] Pallets Projects. Jinja documentation. <https://jinja.palletsprojects.com/en/stable/>.
- [34] SE4AI, University of Bari. Ct-covid. <https://github.com/se4ai2122-cs-uniba/CT-COVID>. GitHub repository, accessed September 2026.

Acknowledgements

Desidero ringraziare il **Prof. Giulio Mallardi** per avermi seguito durante questo lavoro, per la disponibilità dimostrata e per i consigli che mi hanno aiutato a dare una direzione più chiara alla tesi.

Ringrazio anche il **Prof. Filippo Lanubile** per avermi permesso di continuare i miei studi in questo campo specifico, con costante supporto e motivazione.

Un ringraziamento va ai miei **amici**, che hanno accompagnato questi anni universitari e condiviso con me momenti molto diversi tra loro. Tra studio, pause, giornate semplici e periodi più pesanti, hanno contribuito a rendere questo percorso molto più piacevole.

Ringrazio **mia madre e mio padre** per il sostegno costante e per esserci sempre stati lungo tutto il percorso, consentendomi di intraprendere e concludere questa nuova avventura e spronandomi sempre a dare di più.

Infine, voglio dedicare un ringraziamento anche a **me stesso**. Per aver continuato anche nei momenti in cui sarebbe stato più semplice rimandare, rallentare o perdere di vista l'obiettivo. Arrivare fino a qui non cancella le difficoltà del percorso, ma dà loro un senso diverso. Rende l'intero percorso un bel ricordo e un'avventura che rimarrà sempre nei miei pensieri.